



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA
**Dipartimento di Informatica, Sistemistica e
Comunicazione**
Corso di Laurea in Informatica

Strategia di apprendimento per il controllo autonomo di un quadricottero 2D

Svolto da:
Michele Minuti (MATR. 879391) e Davide Albuzzi (MATR. 879363)

Sistemi complessi: modelli e simulazioni
Anno Accademico 2025-2026

Abstract

In questo lavoro vengono confrontate diverse strategie di controllo per il volo autonomo di un quadricottero in un ambiente simulato bidimensionale. In particolare, vengono analizzati un controllore PID tradizionale e quattro algoritmi di Reinforcement Learning (A2C, PPO e SAC) utilizzando l'ambiente **BalloonEnv**, caratterizzato dalla presenza di vincoli fisici realistici alla quale è stato poi aggiunto perturbazioni ambientali quali vento e rumore sensoriale (**BalloonEnv_noisy**).

È stato inoltre studiato l'impatto del *Curriculum Learning* sulla stabilità e sulla capacità di adattamento degli agenti. I risultati mostrano che SAC rappresenta l'approccio più efficace in termini di rapidità di convergenza e prestazioni finali, mentre PPO si distingue per robustezza e affidabilità, beneficiando significativamente dell'addestramento progressivo. Al contrario, A2C evidenzia maggiori difficoltà nel gestire ambienti stocastici complessi. Lo studio mette in evidenza vantaggi e limiti delle diverse tecniche considerate per il controllo autonomo di sistemi dinamici soggetti a disturbi.

Indice

1	Introduzione	4
2	Controllo classico: PID	6
2.1	Principio di funzionamento	6
2.2	Struttura del controllore PID	7
2.2.1	Azione proporzionale (P)	7
2.2.2	Azione integrale (I)	7
2.2.3	Azione derivativa (D)	7
2.3	Comportamento complessivo	8
2.4	Tuning dei parametri	9
2.5	Limiti del controllo PID	9
2.6	Interpretazione intuitiva (applicata al drone)	10
2.7	Implementazione nel progetto	10
2.8	Conclusione	12
3	Reinforcement Learning e Curriculum Learning	13
3.1	Il Paradigma del Reinforcement Learning	13
3.2	Tassonomia e Classificazione degli Algoritmi	14
3.2.1	Model-Based vs Model-Free	14
3.2.2	Value-Based, Policy-Based e Actor-Critic	14
3.2.3	On-Policy vs Off-Policy	15
3.3	Curriculum Learning	15
3.3.1	Vantaggi e Limitazioni del Curriculum Learning	16
4	Ambiente	17
4.1	Dinamica fisica	17
4.2	Arena di valutazione (BalloonEnv)	19
4.3	Ambiente di addestramento (droneEnv)	21
4.3.1	Spazio delle azioni	21
4.3.2	Frame-skip	21
4.3.3	Spazio delle osservazioni	21
4.3.4	Funzione di reward	22
4.3.5	Episodi e terminazione	22
4.4	Perturbazioni: vento e rumore sensoriale	23
4.4.1	Modello del vento	23
4.4.2	Rumore sui sensori	24
4.4.3	Confronto sintetico tra ambiente nominale e perturbato	25

5	Agenti	26
5.1	Panoramica	26
5.2	Agente Human	27
5.3	Agente PID	27
5.3.1	Osservazione	27
5.3.2	Cascata di controllo	27
5.3.3	Guadagni	28
5.4	Agente DQN (storico)	28
5.5	Agenti a politica continua: SAC, A2C, PPO	29
5.5.1	Osservazione e azione	29
5.5.2	Caricamento dei modelli addestrati	29
5.6	Procedure di addestramento	30
5.6.1	Addestramento nominale	30
5.6.2	Addestramento su ambiente perturbato	30
5.6.3	Curriculum learning	30
6	Esperimenti e risultati	32
6.1	Protocollo di valutazione e metriche	32
6.1.1	Metrica di valutazione	32
6.1.2	Metriche derivate	33
6.1.3	Configurazioni valutate	33
6.2	Hyperparameter Tuning	33
6.3	Risultati in ambiente nominale	35
6.3.1	A2C in ambiente nominale	36
6.3.2	PPO in ambiente nominale	37
6.3.3	SAC in ambiente nominale	38
6.4	Robustezza in ambiente perturbato	38
6.4.1	A2C in ambiente perturbato	39
6.4.2	PPO in ambiente perturbato	40
6.4.3	SAC in ambiente perturbato	41
6.5	Impatto del Curriculum Learning	41
6.5.1	A2C: il curriculum non è sufficiente	42
6.5.2	PPO: riduzione della varianza	45
6.5.3	SAC: curriculum poco rilevante	47
6.6	Confronto quantitativo: palloncini raccolti	49
6.6.1	Risultati in ambiente nominale	49
6.6.2	Risultati in ambiente perturbato	50
6.7	Analisi per algoritmo	51
6.7.1	A2C	51
6.7.2	PPO	51
6.7.3	SAC	52
6.7.4	Confronto PID vs RL	52
7	Conclusioni e sviluppi futuri	54
7.1	Sintesi dei risultati	54
7.2	Limiti del lavoro	55
7.3	Sviluppi futuri	56

Capitolo 1

Introduzione

La presente relazione descrive il lavoro svolto per la realizzazione del progetto relativo al corso di *Sistemi Complessi: Modelli e Simulazioni* dell'Università degli Studi di Milano-Bicocca.

L'obiettivo del progetto è stato lo sviluppo e l'analisi comparativa di strategie di controllo per il volo autonomo di un quadricottero in ambiente simulato 2D, confrontando il controllore PID classico con moderni algoritmi di Reinforcement Learning (RL): A2C, PPO e SAC. In tale contesto, sono stati aggiunti e studiati gli effetti delle perturbazioni ambientali (vento casuale, rumore sensori) sulla robustezza delle policy e l'impatto del *curriculum learning* sulla convergenza e generalizzazione degli agenti RL.

Sono state esaminate diverse configurazioni sperimentali: controllo in ambiente nominale “pulito”, ambienti perturbati per test di robustezza, e strategie di addestramento con e senza curriculum progressivo che incrementa gradualmente la difficoltà delle perturbazioni.

Per la realizzazione del progetto, è stato adottato l'ambiente di simulazione **BalloonEnv** (basato su framework Gym), che modella la dinamica 2D di un quadricottero che deve raggiungere e “pungere” un palloncino target, soggetto a vincoli fisici realistici e perturbazioni controllate. La scelta di una simulazione 2D si basa sulla volontà di concentrarsi sugli aspetti algoritmici di controllo e robustezza, mantenendo una complessità computazionale accessibile per analisi statistiche estensive su multiple *seed* e parametri.

La relazione è organizzata nei seguenti capitoli:

- **Controllo classico: PID:** richiamo teorico del controllore PID e descrizione della sua implementazione specifica per il quadricottero 2D.
- **Reinforcement Learning:** introduzione teorica essenziale e panoramica degli algoritmi implementati (DQN, A2C, PPO, SAC), con particolare attenzione al *curriculum learning*.
- **Ambiente:** descrizione dettagliata di BalloonEnv nelle sue varianti nominale e perturbata, inclusa la funzione di reward e i vincoli fisici.

- **Agente:** analisi dello spazio delle osservazioni (posizione, velocità, angoli relativi al target) e delle azioni (thrust totale e differenziale).
- **Esperimenti e risultati:** setup sperimentale completo (parametri di training, hardware, numero di run), confronti quantitativi PID vs RL, analisi di robustezza e impatto del curriculum.
- **Conclusioni e sviluppi futuri:** sintesi dei risultati principali e proposta di estensioni.

Capitolo 2

Controllo classico: PID

Il controllo Proporzionale-Integrale-Derivativo (PID) è uno dei metodi più utilizzati nei sistemi di controllo automatico. La sua diffusione deriva dalla semplicità di implementazione e dalla buona efficacia in una vasta gamma di sistemi dinamici, anche complessi.

Nel contesto di questo lavoro, il controllore PID viene utilizzato come *baseline classica* per il controllo del drone in ambiente 2D, al fine di confrontarlo con approcci moderni basati su Reinforcement Learning.

2.1 Principio di funzionamento

Un controllore PID opera in retroazione (feedback): misura lo stato attuale del sistema, lo confronta con un valore desiderato (setpoint) e calcola un errore:

$$e(t) = r(t) - y(t) \quad (2.1)$$

dove:

- $r(t)$ è il riferimento (posizione del target),
- $y(t)$ è l'uscita del sistema (posizione del drone),
- $e(t)$ è l'errore di controllo.

L'obiettivo del controllore è ridurre l'errore fino a zero, regolando l'input al sistema.

2.2 Struttura del controllore PID

Il segnale di controllo $u(t)$ è dato dalla somma di tre contributi:

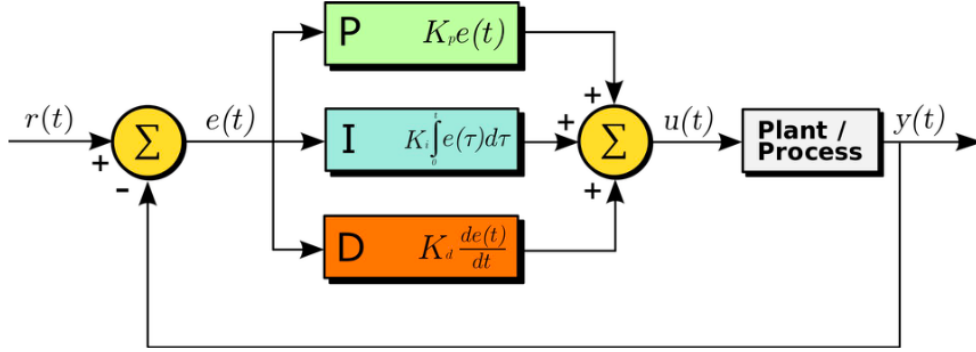


Figure 2.1: Schema a blocchi di un controllore PID
[3]

2.2.1 Azione proporzionale (P)

L'azione proporzionale reagisce all'errore attuale:

$$K_P e(t) \quad (2.2)$$

Un valore elevato di K_P rende il sistema più reattivo, ma può introdurre oscillazioni o instabilità. Un valore troppo basso, invece, rende la risposta lenta e poco efficace.

Intuizione: è come una correzione immediata basata sulla distanza dal target.

2.2.2 Azione integrale (I)

L'azione integrale considera la storia dell'errore:

$$K_I \int_0^t e(\tau) d\tau \quad (2.3)$$

Questa componente permette di eliminare errori persistenti nel tempo (errore a regime), garantendo che il sistema raggiunga esattamente il valore desiderato.

Intuizione: se il drone rimane leggermente distante dal target per molto tempo, questa azione “accumula” l'errore e forza la correzione.

Tuttavia, un uso eccessivo può causare *overshoot* e fenomeni di *windup*.

2.2.3 Azione derivativa (D)

L'azione derivativa anticipa il comportamento del sistema osservando la variazione dell'errore:

$$K_D \frac{de(t)}{dt} \quad (2.4)$$

Questa componente migliora la stabilità riducendo oscillazioni e sovraelongazioni.

Intuizione: agisce come un “freno predittivo” che si attiva quando il sistema si sta avvicinando troppo velocemente al target.

Tuttavia, è molto sensibile al rumore e spesso viene ridotta o rimossa in applicazioni pratiche.

2.3 Comportamento complessivo

Il controllore PID combina le tre azioni per ottenere un comportamento bilanciato:

- la parte P reagisce all’errore attuale,
- la parte I elimina l’errore accumulato,
- la parte D stabilizza la risposta.

La qualità del controllo dipende fortemente dai parametri K_P , K_I e K_D , che vengono tipicamente regolati tramite tecniche di tuning.

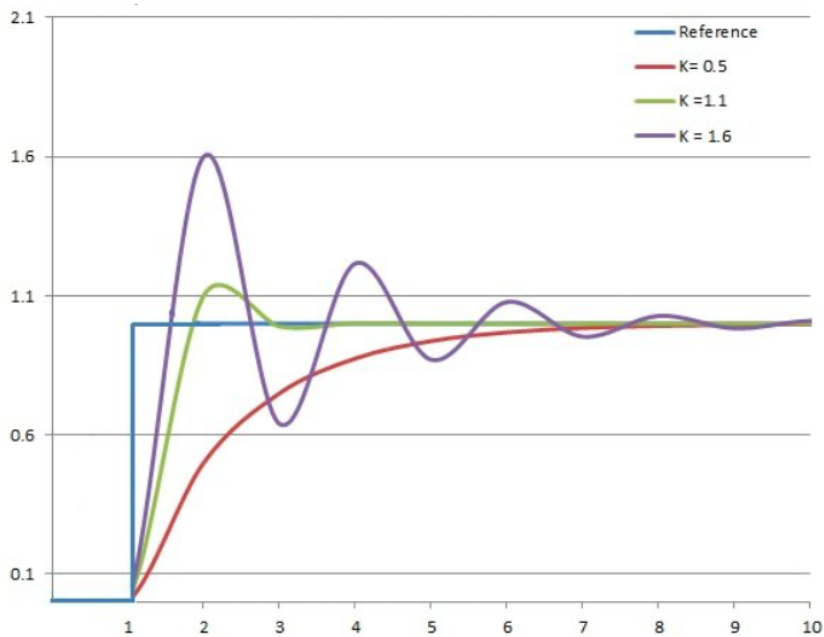


Figure 2.2: Risposta a gradino del sistema per diversi valori di K_P (con K_I e K_D costanti) [3]

2.4 Tuning dei parametri

Non esiste una configurazione universale ottimale per i parametri PID. Una delle tecniche più note è il metodo di Ziegler-Nichols, che permette di ottenere valori iniziali ragionevoli basati sulla risposta del sistema.

- Si imposta inizialmente il controllore con sola azione proporzionale, ponendo $K_I = 0$ e $K_D = 0$.
- Si aumenta gradualmente il guadagno proporzionale K fino a ottenere oscillazioni sostenute dell'uscita.
- Il valore di K per cui compaiono oscillazioni permanenti è detto *guadagno critico* K_u .
- Si misura il *periodo critico* P_u , cioè il periodo delle oscillazioni sostenute.
- A partire da K_u e P_u , si calcolano i parametri del controllore P, PI o PID secondo le tabelle del metodo.

Metodo Ziegler–Nichols			
Tipo	K_p	τ_i	τ_d
P	$0,50K_u$	-	-
PI	$0,45K_u$	$P_u/1,2$	-
PID	$0,60K_u$	$P_u/2$	$P_u/8$

Figure 2.3: Metodo Ziegler–Nichols [10]

In alternativa, è comune procedere per tentativi (trial and error), osservando la risposta del sistema e regolando:

- K_P per la reattività,
- K_I per eliminare l'errore a regime,
- K_D per ridurre oscillazioni.

2.5 Limiti del controllo PID

Nonostante la sua efficacia, il PID presenta alcune limitazioni importanti:

- non si adatta automaticamente a cambiamenti significativi del sistema,
- può diventare instabile in presenza di forti perturbazioni o rumore,
- è progettato per sistemi a singola variabile (SISO),
- richiede una fase di tuning manuale.

Nel contesto del BalloonEnv, queste limitazioni diventano evidenti in presenza di vento e rumore sensoriale, dove il sistema può perdere stabilità o diventare meno efficiente.

2.6 Interpretazione intuitiva (applicata al drone)

Nel nostro caso di studio, il PID può essere interpretato come un sistema che:

- spinge il drone verso il target (azione proporzionale),
- corregge errori persistenti dovuti al vento (azione integrale),
- smorza oscillazioni dovute a correzioni troppo aggressive (azione derivativa).

Questo lo rende un buon punto di riferimento per valutare le prestazioni degli algoritmi di Reinforcement Learning, che invece apprendono una politica di controllo direttamente dall'interazione con l'ambiente.

2.7 Implementazione nel progetto

Il controllore PID implementato in questo progetto si ispira al repository `Quadcopter-AI` di Alexandre Sajus [8], adattato a una dinamica bidimensionale. L'architettura segue una struttura a quattro controllori PID in cascata, organizzati in due rami principali: controllo orizzontale (asse X) e controllo verticale (asse Y).

Nel ramo orizzontale, l'errore di posizione lungo l'asse X viene elaborato da un primo PID per generare un angolo di roll desiderato. Tale riferimento viene confrontato con l'angolo attuale del drone e inviato a un secondo PID, che produce una coppia differenziale tra i due rotori, responsabile dell'inclinazione e quindi dello spostamento laterale.

Nel ramo verticale, l'errore di altezza (asse Y) viene utilizzato per calcolare una velocità verticale desiderata tramite un PID. Successivamente, la differenza tra velocità verticale desiderata e reale viene fornita a un secondo PID, che determina la spinta media da applicare ai rotori, controllando così la quota del drone.

Il nucleo del sistema è rappresentato dalla classe `PID`, che implementa un controllore discreto con componenti proporzionale, integrale e derivativa, oltre a un meccanismo di saturazione dell'output:

```

1 class PID:
2     def __init__(self, KP, KI, KD, saturation_max,
3         saturation_min):
4         self.kp = KP
5         self.ki = KI
6         self.kd = KD
7         self.error_last = 0
8         self.integral_error = 0
9         self.saturation_max = saturation_max
10        self.saturation_min = saturation_min
11
12    def compute(self, error, dt):
13        derivative_error = (error - self.error_last) / dt
14        self.integral_error += error * dt
15        output = (self.kp * error +
16            self.ki * self.integral_error +
17            self.kd * derivative_error)
18        self.error_last = error
19        # Saturazione
20        if output > self.saturation_max:
21            output = self.saturation_max
22        elif output < self.saturation_min:
23            output = self.saturation_min
24        return output

```

Listing 2.1: Implementazione PID discreta

I quattro controllori PID sono istanziati nella classe `PIDPlayer` con parametri ottimizzati empiricamente tramite il metodo Ziegler–Nichols mostrato in figura 2.3:

- `xPID = PID(0.2, 0, 0.2, 25, -25)`: errore di posizione orizzontale → angolo di roll desiderato.
- `aPID = PID(0.02, 0, 0.01, 1, -1)`: errore di roll → coppia differenziale tra i rotori.
- `yPID = PID(2.5, 0, 1.5, 100, -100)`: errore di altezza → velocità verticale desiderata.
- `ydPID = PID(1, 0, 0, 1, -1)`: errore di velocità verticale → spinta media.

Il sistema opera con un passo temporale discreto pari a $dt = \frac{1}{60}$ s (60 Hz). Le azioni finali consistono nella combinazione di una componente di spinta media (derivata dal controllo verticale) e di una componente differenziale (derivata dal controllo angolare), applicate ai due rotori. La presenza di limiti di saturazione sugli output dei PID contribuisce a evitare comandi eccessivi e a migliorare la stabilità del sistema.

2.8 Conclusione

Il controllore PID rappresenta una soluzione semplice ma efficace per il controllo di sistemi dinamici. Tuttavia, la sua natura rigida e la dipendenza da una corretta taratura lo rendono meno adatto a scenari complessi e altamente stocastici come quelli simulati nel BalloonEnv.

Per questo motivo, nel capitolo successivo verranno introdotti metodi di Reinforcement Learning, che permettono di apprendere strategie di controllo adattive direttamente dall'esperienza.

Capitolo 3

Reinforcement Learning e Curriculum Learning

Il panorama dell'apprendimento automatico si divide tradizionalmente in tre grandi paradigmi: l'apprendimento supervisionato, l'apprendimento non supervised e il **Reinforcement Learning** (RL). Mentre i primi due si basano rispettivamente sulla presenza di un dataset etichettato o sulla ricerca di pattern intrinseci nei dati, il Reinforcement Learning si fonda su un concetto diverso: l'apprendimento guidato dall'interazione diretta con l'ambiente secondo un meccanismo di prova ed errore (*trial-and-error*).

3.1 Il Paradigma del Reinforcement Learning

Il Reinforcement Learning si definisce come un framework matematico in cui un'entità computazionale autonoma, denominata **agente**, impara a compiere decisioni sequenziali all'interno di un *ambiente* dinamico con l'obiettivo di massimizzare un segnale di ricompensa cumulativo. L'agente non viene istruito su quali azioni intraprendere, ma deve scoprire autonomamente quali strategie producono il massimo beneficio nel lungo periodo.

L'interazione fondamentale tra l'agente e l'ambiente si sviluppa in cicli iterativi:

- **Ambiente:** Il contesto dinamico esterno all'agente. Esso fornisce gli stimoli e le informazioni necessarie per determinare lo stato corrente del sistema.
- **Agente:** Il componente decisionale che riceve lo stato in input, analizza il contesto operativo e seleziona un'**azione** in grado di influenzare l'ambiente.
- **Reward (Ricompensa):** Il feedback scalare restituito dall'ambiente in risposta all'azione eseguita. Può essere positivo o negativo e quantifica il successo immediato dell'azione rispetto all'obiettivo finale.

Attraverso questa continua alternanza di percezione, azione e ricompensa, l'agente ottimizza una strategia di comportamento interna denominata **policy**.

La policy rappresenta la funzione che associa a ciascuno stato la scelta dell'azione ottimale da intraprendere.

La formulazione dell'azione generata dall'agente può strutturarsi secondo due modalità, definite dalla natura geometrica dello spazio delle azioni:

- **Discreta:** L'insieme dei comandi consentiti è finito e numerabile. L'agente seleziona un'opzione netta all'interno di un elenco di alternative predefinite. Questa configurazione è tipica di approcci basati sulla funzione valore come l'algoritmo DQN. [6]
- **Continua:** L'azione è espressa attraverso un vettore di valori reali compresi all'interno di un intervallo numerico privo di interruzioni. Questa modalità consente un controllo preciso e fluido, rappresentando la scelta d'elezione per algoritmi di tipo actor-critic quali PPO [9], SAC [4] e A2C [5].

3.2 Tassonomia e Classificazione degli Algoritmi

Gli algoritmi di Reinforcement Learning possono essere mappati e classificati seguendo diversi criteri teorici, a seconda della conoscenza dell'ambiente, dei meccanismi di aggiornamento e della gestione dei dati campionari [11].

3.2.1 Model-Based vs Model-Free

La prima grande distinzione mappa gli algoritmi in base all'utilizzo o alla stima della dinamica fisica dell'ambiente circostante:

- **Model-Based:** I metodi *model-based* presuppongono la costruzione o l'utilizzo di un modello matematico esplicito del sistema per prevedere le transizioni di stato e le ricompense. Questo permette all'agente di pianificare le proprie scelte (*planning*) prima ancora di eseguirle nel mondo reale. Sebbene efficienti, la loro applicabilità è limitata dalla difficoltà di modellare analiticamente sistemi complessi o fortemente stocastici.
- **Model-Free:** Al contrario, gli approcci *model-free* non richiedono alcuna rappresentazione matematica della dinamica ambientale. L'agente effettua una ricerca diretta della policy ottima basandosi esclusivamente sull'esperienza empirica accumulata. Questa classe di algoritmi — a cui appartengono soluzioni quali DQN [6], A2C [5], PPO [9] e SAC [4] — si rivela ideale nei contesti non lineari e perturbati, dove una modellazione analitica risulterebbe parziale o intrinsecamente imprecisa.

3.2.2 Value-Based, Policy-Based e Actor-Critic

Nell'ambito dei metodi *model-free*, l'ottimizzazione della strategia può seguire criteri filosofici differenti, che governano i quattro modelli presi in esame in questo lavoro:

- **Value-Based (Metodi basati sul valore):** Questi algoritmi ottimizzano la strategia dell'agente in modo indiretto. Invece di calcolare subito l'azione da compiere, stimano il valore di ogni possibile scelta in un dato

stato tramite una funzione di valore (Q). L'agente sceglierà poi l'azione che promette la ricompensa a lungo termine più alta. Questo approccio costituisce il nucleo di **DQN (Deep Q-Network)**, l'algoritmo a spazio d'azione discreto utilizzato in questo lavoro come termine di paragone fondamentale rispetto ai metodi continui. [6]

- **Policy-Based (Metodi basati sulla policy):** Invece di stimare il valore delle azioni, i metodi *policy-based* ottimizzano in modo diretto i parametri della policy attraverso algoritmi di salita del gradiente (*policy gradient*). Questo approccio consente di gestire nativamente spazi di azione complessi e continui. Sebbene i metodi di gradiente puri soffrano storicamente di un'elevata varianza, questa impostazione rappresenta il pilastro concettuale dei moderni algoritmi di controllo continuo.
- **Actor-Critic (Metodi Attore-Critico):** Rappresentano la fusione dei due mondi precedenti, combinando i vantaggi dei metodi basati sulla policy con la stabilità di quelli basati sul valore. L'architettura prevede due componenti neurali distinte: l'**Actor** (Attore), che aggiorna direttamente la policy per selezionare le azioni, e il **Critic** (Critico), che valuta l'efficacia delle decisioni prese dall'Attore fornendo un feedback che riduce la varianza degli aggiornamenti. Questa struttura costituisce l'asse portante dei modelli continui testati in questo lavoro: **A2C (Advantage Actor-Critic)** [5], **PPO (Proximal Policy Optimization)** [9] e **SAC (Soft Actor-Critic)** [4].

3.2.3 On-Policy vs Off-Policy

La gestione e il riutilizzo dei dati campionari distingue infine gli algoritmi in due macro-categorie [11]:

- **On-Policy:** Valutano e migliorano esclusivamente la stessa policy utilizzata dall'agente per interagire con l'ambiente in quel preciso istante. L'esperienza raccolta è strettamente legata alla strategia corrente e non può essere riutilizzata una volta aggiornati i parametri.
- **Off-Policy:** Possono ottimizzare una policy target differente rispetto alla policy comportamentale impiegata per l'esplorazione. Questa proprietà consente all'agente di memorizzare l'esperienza passata all'interno di memorie storiche (*replay buffer*) e di sfruttare dati generati da strategie precedenti, massimizzando l'efficienza campionaria.

3.3 Curriculum Learning

L'applicazione del Reinforcement Learning in ambienti complessi si scontra frequentemente con il problema della **reward sparsity** (scarsità delle ricompense). Quando un agente viene inserito da zero in un contesto difficile, le probabilità di completare con successo il compito tramite azioni casuali sono prossime allo zero. Di conseguenza, l'agente riceve solo feedback nulli o penalità, bloccando l'apprendimento all'interno di soluzioni sub-ottimali.

Per superare questa limitazione si introduce il paradigma del **Curriculum Learning**, un approccio metodologico che prende ispirazione dai processi educativi della pedagogia umana. Di fronte a una disciplina complessa, un essere umano inizia affrontando i concetti base per poi incrementare la difficoltà in modo graduale [1].

Nel contesto del Reinforcement Learning, il Curriculum Learning organizza l'addestramento suddividendo il compito principale in una sequenza ordinata di **sotto-task** (o lezioni) a complessità incrementale:

1. **Fase Iniziale:** L'agente viene inserito in un ambiente semplificato, strutturato per facilitare la scoperta spontanea delle prime ricompense e l'apprendimento dei comportamenti motori e logici di base.
2. **Progressione Didattica:** Una volta soddisfatto un determinato criterio di performance quantitativo sulla lezione attuale, l'ambiente evolve introducendo nuovi vincoli, ostacoli o dinamiche più complesse.
3. **Fase Finale:** L'agente, avendo progressivamente consolidato la propria struttura di policy, viene infine introdotto nell'ambiente completo per terminare l'addestramento.

3.3.1 Vantaggi e Limitazioni del Curriculum Learning

L'adozione di una strategia a curriculum offre importanti benefici teorici, tra cui un netto incremento della velocità di convergenza algoritmica, la possibilità di risolvere task altrimenti impraticabili da zero e un miglioramento della robustezza della policy finale [1].

Tuttavia, la metodologia presenta alcune limitazioni critiche dal punto di vista ingegneristico:

- **Necessità di conoscenza del dominio:** La scomposizione del compito principale in sotto-task non è automatica e richiede una profonda conoscenza analitica del sistema da parte dello sviluppatore.
- **Rischio di Overfitting e Bias:** Un'esposizione prolungata a scenari intermedi può causare un fenomeno di *overfitting* sulle singole lezioni [1]. L'agente rischia di specializzare la propria policy su dinamiche parziali, faticando a generalizzare quando i sotto-task vengono presentati in sequenza nel circuito finale.
- **Bias di Ordinamento:** L'ordine sequenziale e i criteri stabiliti per il superamento delle lezioni influenzano pesantemente la trajectory di ottimizzazione della rete neurale, potendo introdurre comportamenti imprevisti o sbilanciati sul lungo periodo.

Capitolo 4

Ambiente

In questo capitolo viene descritto l'ambiente di simulazione utilizzato per addestrare e confrontare i diversi controllori (HUMAN, PID, A2C, PPO, SAC). Il progetto si basa su due simulatori distinti, condividenti la stessa dinamica fisica ma con scopi differenti:

- un'arena di valutazione (`balloon.py` / `balloon_noisy.py`), in cui più agenti, visualizzati a schermo, competono in tempo reale per raccogliere il maggior numero di target entro un tempo limite;
- un ambiente di addestramento compatibile con l'interfaccia Gym (`droneEnv`, definito in `env.py` ed `env_noisy.py`), utilizzato per allenare gli agenti di Reinforcement Learning tramite la libreria `stable-baselines3` [7].

Entrambi i simulatori sono disponibili in una versione *nominale*, priva di disturbi, e in una versione *perturbata*, che introduce vento e rumore sensoriale per lo studio della robustezza.

La descrizione degli agenti che operano in questi ambienti (incluse le specifiche osservazioni ricevute da ciascun tipo di controllore e le rispettive procedure di addestramento) è rimandata al Capitolo 5.

4.1 Dinamica fisica

Il drone è modellato come un corpo rigido che si muove in un piano 2D. Il suo stato è completamente descritto da:

- posizione (x, y) ,
- velocità lineare (v_x, v_y) ,
- orientamento θ (angolo rispetto alla verticale),
- velocità angolare ω .

Il sistema di attuazione è composto da due propulsori indipendenti, sinistro e destro, che generano rispettivamente le spinte scalari F_L ed F_R . Queste forze

determinano sia il moto traslazionale (spostamento nello spazio) sia il moto rotazionale (variazione dell'orientamento).

Le accelerazioni del drone derivano direttamente dalle leggi di Newton e dalla scomposizione delle forze lungo gli assi orizzontale e verticale, come descritto nel lavoro di riferimento [8]. In particolare, la somma delle spinte $F_L + F_R$ controlla il movimento complessivo, mentre la loro differenza $F_L - F_R$ genera una coppia che induce rotazione.

$$a_x = -\frac{F_L + F_R}{m} \sin \theta + w_x, \quad (4.1)$$

$$a_y = g - \frac{F_L + F_R}{m} \cos \theta + w_y, \quad (4.2)$$

$$\alpha = \frac{\ell (F_R - F_L)}{m}, \quad (4.3)$$

dove g è l'accelerazione di gravità, m la massa del drone, ℓ il braccio tra il baricentro e i propulsori, e (w_x, w_y) la componente di accelerazione dovuta al vento (nulla nella versione nominale, si veda la Sezione 4.4).

Per l'evoluzione temporale del sistema si utilizza uno schema di integrazione numerica di tipo *Euler semi-implicito* (o symplettico): la velocità viene aggiornata utilizzando l'accelerazione corrente, e la posizione viene aggiornata utilizzando la velocità già aggiornata,

$$v_{t+1} = v_t + a_t, \quad x_{t+1} = x_t + v_{t+1}, \quad (4.4)$$

e analogamente per la coppia (ω, θ) .

$$\omega_{t+1} = \omega_t + \alpha_t, \quad \theta_{t+1} = \theta_t + \omega_{t+1}. \quad (4.5)$$

Questo schema garantisce una maggiore stabilità numerica rispetto all'Euler esplicito a fronte di un costo computazionale identico.

Il passo di integrazione corrisponde a un singolo frame di simulazione (1/60 s) e non compare esplicitamente nelle equazioni: tutte le costanti fisiche sono pertanto calibrate empiricamente in queste unità “per frame” piuttosto che in unità SI.

Le costanti fisiche, identiche nei due simulatori, sono riassunte in Tabella 4.1.

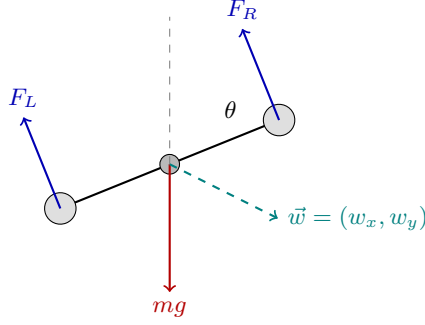


Figure 4.1: Schema delle forze agenti sul drone: spinte dei propulsori F_L , F_R , gravità mg e, nella versione perturbata, accelerazione dovuta al vento $\vec{w}$.

Costante	Valore	Descrizione
g	0.08	Accelerazione di gravità (per frame)
m	1	Massa del drone
ℓ	25 px	Braccio tra baricentro e propulsori
<code>thruster_mean</code>	0.04	Spinta di riferimento per il volo stazionario
<code>thruster_amplitude</code>	0.04	Escursione massima di spinta verticale
<code>diff_amplitude</code>	0.003	Escursione massima di spinta differenziale (controllo di rotazione)

Table 4.1: Costanti fisiche del simulatore (ambiente nominale).

4.2 Arena di valutazione (BalloonEnv)

L'arena di valutazione rappresenta un piano cartesiano di 800×800 pixel su cui agisce un drone soggetto a gravità e controllato da due propulsori. All'inizio di ogni partita vengono generate 100 posizioni target casuali all'interno della regione centrale $[200, 600] \times [200, 600]$ px; tali posizioni sono condivise da tutti i controllori attivi nella stessa simulazione, in modo che il confronto tra agenti diversi avvenga sulla medesima sequenza di obiettivi.

Il simulatore avanza a 60 fps e ogni partita ha una durata fissa di 100 secondi simulati (circa 6000 passi), al termine dei quali viene confrontato il punteggio di ciascun agente, definito come il numero di target raggiunti.

Ad ogni passo viene calcolata la distanza euclidea d tra il drone e il target corrente. Si distinguono due condizioni:

- **Target raggiunto:** se $d < 50$ px, il contatore di target dell'agente viene incrementato e l'agente procede verso il target successivo nella sequenza pre-generata.
- **Divergenza (“crash”):** se $d > 1000$ px, l'agente viene considerato “morto” e congelato per un tempo fisso di `respawn_timer` = 3 secondi,

trascorso il quale viene ripristinato al centro dello schermo (400, 400) con velocità lineare, velocità angolare e angolo nulli. Il contatore di target non viene azzerato.

L'architettura del codice consente di eseguire più controllori in parallelo all'interno della stessa finestra di simulazione, ciascuno con il proprio stato fisico e il proprio contatore di target, ma soggetto allo stesso orologio di simulazione e, nella variante perturbata, allo stesso campo di vento istantaneo.

Questa configurazione consente di osservare direttamente gli agenti all'opera all'interno della simulazione, rendendo possibile un'analisi qualitativa dettagliata dei loro comportamenti. In particolare, permette di valutare come ciascun controllore reagisce a diverse situazioni, come il raggiungimento dei target, le fasi di instabilità o le perturbazioni ambientali, evidenziando differenze nel controllo, nella stabilità e nelle strategie di movimento.

I confronti quantitativi finali, invece, sono stati effettuati separatamente eseguendo esperimenti controllati direttamente sull'ambiente di training, al fine di garantire maggiore coerenza con il processo di apprendimento e una valutazione statisticamente più robusta.

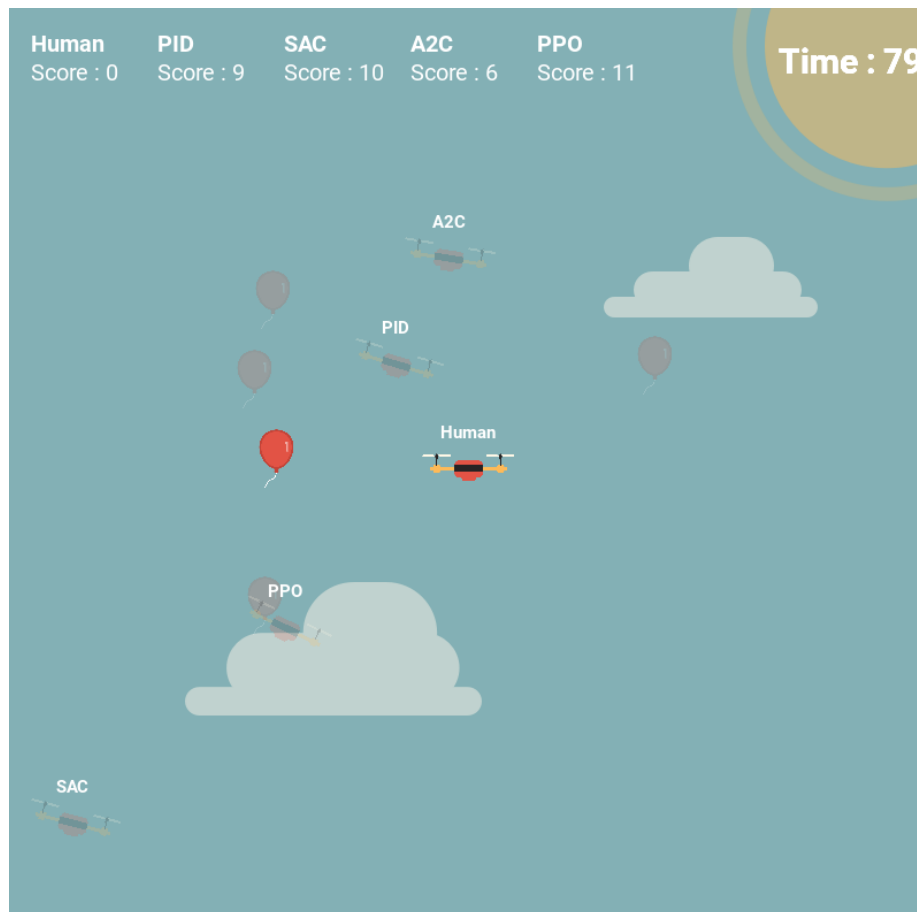


Figure 4.2: Arena di valutazione (BalloonEnv)

4.3 Ambiente di addestramento (droneEnv)

Gli agenti di Reinforcement Learning (SAC, A2C, PPO e le rispettive varianti) non vengono addestrati direttamente nell'arena di valutazione, bensì in un ambiente separato, **droneEnv**, conforme all'interfaccia `gym.Env` [2] e quindi compatibile con gli algoritmi di **stable-baselines3**. Questo ambiente condiziona la stessa dinamica fisica della Sezione 4.1 ma definisce un proprio spazio di azione, un proprio spazio di osservazione e una propria funzione di reward, pensati per agevolare l'apprendimento.

4.3.1 Spazio delle azioni

L'azione fornita dalla policy è una coppia di valori continui normalizzati ($action_0, action_1$) $\in [-1, 1]^2$. Questi vengono convertiti nelle spinte dei due propulsori secondo

$$F_L = \text{thruster_mean} + action_0 \cdot \text{thruster_amplitude} + action_1 \cdot \text{diff_amplitude}, \quad (4.6)$$

$$F_R = \text{thruster_mean} + action_0 \cdot \text{thruster_amplitude} - action_1 \cdot \text{diff_amplitude}, \quad (4.7)$$

dove $action_0$ controlla la spinta media (salita/discesa) e $action_1$ la spinta differenziale (rotazione). Questa stessa equazione, una volta addestrata la policy, viene riutilizzata anche nell'arena di valutazione dagli agenti RL (si veda il Capitolo 5).

4.3.2 Frame-skip

Ad ogni decisione, l'azione scelta dalla policy viene mantenuta costante e applicata per 5 frame fisici consecutivi (*frame-skip*), prima che venga richiesta una nuova decisione. Poiché la fisica avanza a 60 fps, questo equivale a circa 12 decisioni al secondo per l'agente, un compromesso che riduce la frequenza di decisione (rendendo il problema più trattabile per la policy) senza alterare la risoluzione della simulazione fisica sottostante.

4.3.3 Spazio delle osservazioni

L'ambiente fornisce un vettore di osservazione normalizzato a 6 componenti,

$$o = \left(\theta', v, \omega, \frac{d}{500}, \varphi_t, \varphi_t - \varphi_v \right), \quad (4.8)$$

dove:

- **1.** $\theta' = \theta \times \pi/180$: Angolo di inclinazione del drone (in radianti). Dice quanto il drone è “storto” rispetto alla verticale. Se $\theta = 0^\circ$, il drone è orizzontale; se $\theta = 90^\circ$, è verticale.
- **2.** $v = \sqrt{v_x^2 + v_y^2}$: Velocità totale (modulo). Ignora la direzione, ti dice solo “quanto velocemente si muove” il drone, indipendentemente dove va.

- **3.** ω : Velocità angolare. Quanto velocemente il drone ruota (gradi/sec-ondo). Se ω è alto, il drone sta girando rapidamente.
- **4.** $d/500 = \sqrt{(x_t - x)^2 + (y_t - y)^2}/500$: Distanza dal target normalizzata. La distanza reale divisa per 500 (la massima distanza possibile). Se $= 0.2$, il target è al 20% della distanza massima.
- **5.** $\varphi_t = \text{atan2}(y_t - y, x_t - x)$: Bearing verso il target. La direzione (angolo) in cui devi guardare per vedere il target. È come un “orologio”: 0° = destra, 90° = sotto, 180° = sinistra, 270° = sopra.
- **6.** $\varphi_t - \varphi_v = \text{atan2}(y_t - y, x_t - x) - \text{atan2}(v_y, v_x)$: Scostamento angolare. La differenza tra “dove voglio andare” (φ_t) e “dove sto andando” (φ_v). Se $= 0^\circ$, il drone va esattamente verso il target; se $= 90^\circ$, sta andando perpendicolarmente.

Questo stesso vettore di osservazione, eventualmente corrotto da rumore (Sezione 4.4.2), è quello fornito agli agenti A2C, PPO e SAC sia in addestramento sia in valutazione.

4.3.4 Funzione di reward

Ad ogni frame simulato (quindi 5 volte per ogni decisione presa), il reward è calcolato come somma di quattro contributi:

- $+\frac{1}{60}$ per ogni frame di sopravvivenza, per incentivare l’agente a non terminare l’episodio anzitempo;
- $-\frac{d}{100 \cdot 60}$ ad ogni frame, una penalità proporzionale alla distanza dal target che guida l’agente verso l’obiettivo;
- $+100$ ogni volta che un target viene raggiunto ($d < 50$ px), dopo di che il target viene ricollocato in una nuova posizione casuale;
- -1000 se il drone si allontana eccessivamente dal target ($d > 1000$ px), evento che termina immediatamente l’episodio.

Questa combinazione di reward denso (sopravvivenza e distanza) e reward sparso (raggiungimento del target, crash) è pensata per fornire un segnale di apprendimento continuo pur mantenendo come obiettivo finale la velocità di raccolta dei target.

4.3.5 Episodi e terminazione

A differenza dell’arena di valutazione (durata fissa di 100 s, episodio singolo per partita), gli episodi di addestramento in `droneEnv` hanno un limite di tempo molto più breve, `time_limit = 20` s, raggiunto il quale l’episodio termina regolarmente (`done=True` senza penalità aggiuntiva).

L’episodio termina anticipatamente anche in caso di crash, con l’applicazione della penalità di -1000 descritta sopra. Ad ogni `reset()`, il drone viene riportato al centro dello schermo (400, 400) con velocità e angolo nulli, e viene generata una nuova posizione casuale per il target.

4.4 Perturbazioni: vento e rumore sensoriale

Per lo studio della robustezza, sia l'arena di valutazione sia l'ambiente di addestramento sono disponibili in una variante perturbata, che introduce due tipologie di disturbo assenti nella versione nominale: un campo di vento globale e rumore sulle osservazioni. In `droneEnv` questi disturbi sono parametri opzionali del costruttore (`wind_enabled`, `wind_speed_max`, `sensor_noise_enabled`, `sensor_noise_std`, ...), il che consente di istanziare ambienti con livelli di difficoltà differenti, come sfruttato dal curriculum learning (Capitolo 5).

4.4.1 Modello del vento

Il vento è implementato come un'accelerazione (w_x, w_y) aggiunta direttamente alle accelerazioni traslazionali del drone (Equazioni (4.1) e (4.2)), senza alcun effetto sulla dinamica rotazionale. Il modello combina due meccanismi:

1. **Domain randomization a inizio episodio.** All'avvio di ogni episodio, direzione e intensità del vento vengono campionate uniformemente, $\text{dir} \sim \mathcal{U}(0^\circ, 360^\circ)$ e $\text{speed} \sim \mathcal{U}(\text{speed}_{\min}, \text{speed}_{\max})$, da cui si ricavano le componenti $w_x = \text{speed} \cos(\text{dir})$, $w_y = \text{speed} \sin(\text{dir})$. Questo garantisce che ogni episodio avvenga in condizioni meteorologiche diverse, evitando l'overfitting a un singolo regime di vento. Il valore di default di $\text{speed}_{\max}$ è 0.04 in `droneEnv`.
2. **Variazione lenta nel tempo (random walk).** Ogni 30 passi di simulazione (`wind_update_every`, corrispondenti a 0.5 s), la direzione e l'intensità del vento vengono perturbate tramite un random walk gaussiano, $\Delta \text{dir} \sim \mathcal{N}(0^\circ, 2^\circ)$ e $\Delta \text{speed} \sim \mathcal{N}(0, 0.003)$, con l'intensità vincolata all'intervallo $[\text{speed}_{\min}, \text{speed}_{\max}]$. Il vento risulta quindi non stazionario ma a variazione lenta, in modo da rappresentare un disturbo realistico senza rendere il compito irrisolvibile.

I parametri di default del modello di vento sono riassunti in Tabella 4.2.

Parametro	Valore di default
Direzione iniziale	$\mathcal{U}(0^\circ, 360^\circ)$
Intensità iniziale	$\mathcal{U}(0, 0.04)$
Intervallo di aggiornamento	ogni 30 passi (0.5 s)
Random walk direzione	$\mathcal{N}(0^\circ, 2^\circ)$
Random walk intensità	$\mathcal{N}(0, 0.003)$, troncata in $[\text{speed}_{\min}, \text{speed}_{\max}]$

Table 4.2: Parametri di default del modello di vento in `droneEnv`. I valori esatti di $\text{speed}_{\min}$, $\text{speed}_{\max}$ sono riconfigurabili e vengono effettivamente variati nelle diverse fasi del curriculum learning (Capitolo 5).

Nell'interfaccia grafica dell'arena perturbata, il vento istantaneo è visualizzato tramite una freccia (direzione e modulo) e un valore numerico in km/h, utile per l'analisi qualitativa delle traiettorie durante le sessioni di valutazione.

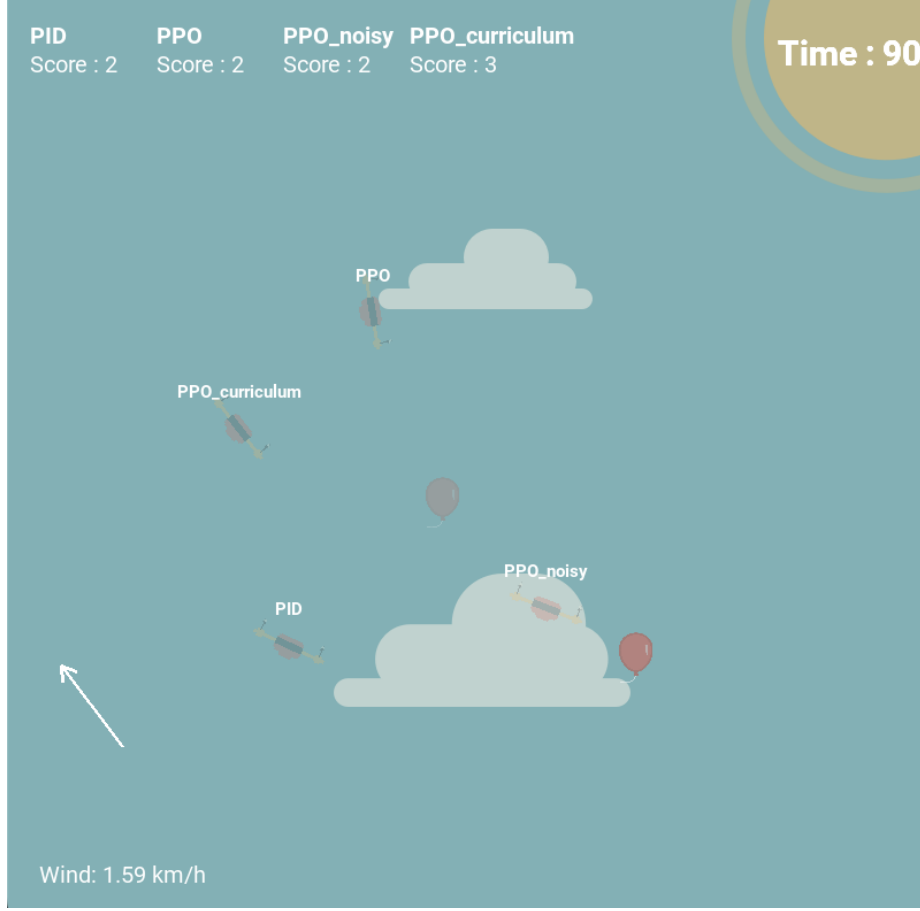


Figure 4.3: Arena di valutazione (BalloonEnv_noisy)

4.4.2 Rumore sui sensori

Per simulare l'imperfezione dei sensori di bordo, il vettore di osservazione normalizzato o (Sezione 4.3.3) può essere corrotto da rumore gaussiano additivo indipendente per componente,

$$\tilde{o} = o + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \Sigma), \quad (4.9)$$

con deviazioni standard differenziate per componente, $\Sigma = \text{diag}(0.01, 0.02, 0.01, 0.02, 0.01, 0.01)$ nella configurazione di default, applicate rispettivamente a $(\theta', v, \omega, d/500, \varphi_t, \varphi_t - \varphi_v)$. Le componenti angolari più sensibili al controllo (orientamento, bearing) ricevono una deviazione standard minore rispetto alle componenti di velocità e distanza. Il vettore Σ è anch'esso un parametro configurabile del costruttore di `droneEnv` (`sensor_noise_std`), e viene infatti scalato a valori più bassi nelle fasi intermedie del curriculum learning.

4.4.3 Confronto sintetico tra ambiente nominale e perturbato

La Tabella 4.3 riassume le differenze principali tra le versioni nominale e perturbata dei due simulatori.

Caratteristica	Versione nominale	Versione perturbata
File (arena)	<code>balloon.py</code>	<code>balloon_noisy.py</code>
File (training)	<code>env.py</code>	<code>env_noisy.py</code>
Vento	Assente	Domain randomization + random walk
Rumore sui sensori	Assente	Gaussiano, Σ per componente
Finalità	Confronto delle prestazioni base	Valutazione della robustezza

Table 4.3: Confronto tra le versioni nominale e perturbata dei due simulatori.

Capitolo 5

Agenti

In questo capitolo vengono descritti con precisione tutti gli agenti confrontati nel progetto: l'agente umano, il controllore PID classico, l'agente storico DQN e gli agenti a politica continua basati su Reinforcement Learning (SAC, A2C, PPO). Per ciascun agente vengono riportate le osservazioni utilizzate e la legge di controllo, e per gli agenti RL la procedura di addestramento effettivamente impiegata.

5.1 Panoramica

Tutti gli agenti condividono la stessa interfaccia, ricevono un'osservazione (`obs`) e restituiscono una coppia di spinte (F_L, F_R).

Ciò che cambia da un agente all'altro è il contenuto di `obs` e la logica interna usata per produrre l'azione. La Tabella 5.1 fornisce una panoramica degli agenti attualmente implementati.

Agente	Tipo	Osservazione	ambiente
Human	Manuale	x	x
PID	Controllo classico	o_{LL}	x
DQN	RL (storico)	o	env_DQN
SAC	RL continuo	o	env_SAC
SAC_noisy	RL continuo	o	env_noisy_SAC
SAC_curriculum	RL continuo	o	env_noisy_SAC
PPO	RL continuo	o	env_PPO
PPO_noisy	RL continuo	o	env_noisy_PPO
PPO_curriculum	RL continuo	o	env_noisy_PPO
A2C	RL continuo	o	env_A2C
A2C_noisy	RL continuo	o	env_noisy_A2C
A2C_curriculum	RL continuo	o	env_noisy_A2C

Table 5.1: Panoramica degli agenti confrontati nel progetto.

5.2 Agente Human

L'agente umano è il più semplice: il giocatore controlla direttamente le spinte dei due propulsori tramite le frecce direzionali della tastiera. Partendo da una spinta di base $F_L = F_R = \text{thruster_mean} = 0.04$, ogni tasto modifica le spinte secondo la Tabella 5.2.

Tasto	Effetto
↑ (Su)	$F_L += 0.04; \quad F_R += 0.04$
↓ (Giù)	$F_L -= 0.04; \quad F_R -= 0.04$
← (Sinistra)	$F_L -= 0.003$
→ (Destra)	$F_R -= 0.003$

Table 5.2: Schema di controllo dell'agente Human.

Questo schema di controllo, per quanto inizialmente poco intuitivo, consente dopo un breve allenamento di navigare tra i target con un numero limitato di crash. L'agente umano viene utilizzato come riferimento di base (baseline), insieme al comando PID, per valutare le prestazioni degli altri controllori.

5.3 Agente PID

L'agente PID si ispira ai controllori di volo utilizzati nei droni reali. Il controllo è determinato dall'errore di posizione fra il drone e il suo target corrente, scomposto in una componente verticale e una orizzontale, ciascuna gestita da una cascata di due controllori PID, per un totale di quattro PID indipendenti, in linea con l'architettura descritta nel paper di riferimento [8].

5.3.1 Osservazione

L'agente PID riceve un'osservazione a basso livello, non normalizzata,

$$o_{LL} = (x_t - x, v_x, y_t - y, v_y, \theta, \omega), \quad (5.1)$$

cioè la posizione del target corrente relativa al drone (componenti chiamate nel codice `error_x` ed `error_y`), le componenti di velocità, l'angolo e la velocità angolare assoluti.

5.3.2 Cascata di controllo

Indichiamo con $e_x = x_t - x$ l'errore orizzontale e con $e_y = y_t - y$ l'errore verticale. La cascata di controllo, implementata nel metodo `act` della classe `PIDPlayer`, opera come segue:

Canale verticale (spinta totale). L'errore verticale viene inviato a un primo PID (`yPID`) che restituisce una velocità verticale desiderata y_c ; l'errore fra questa velocità desiderata e quella corrente viene inviato a un secondo PID (`ydPID`) che restituisce l'ampiezza di spinta $action_0$ da applicare in egual misura ai due propulsori:

$$y_c = \text{yPID}(e_y) \quad (5.2)$$

$$\text{action}_0 = \text{ydPID}(-(y_c - y)) \quad (5.3)$$

Canale orizzontale (rotazione). L'errore orizzontale viene inviato a un primo PID (**xPID**) che restituisce un angolo desiderato θ_c (l'inclinazione necessaria per spostarsi lateralmente verso il target); l'errore fra questo angolo desiderato e quello corrente viene inviato a un secondo PID (**aPID**) che restituisce la spinta differenziale action_1 :

$$\theta_c = \text{xPID}(-e_x) \quad (5.4)$$

$$\text{action}_1 = \text{aPID}(-(\theta_c - \theta)) \quad (5.5)$$

Le due uscite vengono combinate secondo le formule descritte nella sezione 4.3.1

5.3.3 Guadagni

Ogni blocco PID implementa la legge standard proporzionale-integrale-derivativa con uscita saturata entro i limiti specificati. I guadagni, tarati manualmente, sono riportati in Tabella 5.3.

PID	K_p	K_i	K_d	Saturazione uscita
xPID	0.2	0	0.2	$[-25, 25]$
aPID	0.02	0	0.01	$[-1, 1]$
yPID	2.5	0	1.5	$[-100, 100]$
ydPID	1	0	0	$[-1, 1]$

Table 5.3: Guadagni dei quattro controllori PID in cascata.

5.4 Agente DQN (storico)

Il primo tentativo di controllo tramite Reinforcement Learning, documentato nel paper di riferimento del progetto [8], ha utilizzato l'algoritmo DQN (Deep Q-Network) nella sua implementazione di default in `stable-baselines3` [7].

A differenza degli altri agenti RL discussi in questo capitolo, DQN è limitato ad azioni discrete; lo spazio di azione è stato quindi discretizzato in cinque comandi, riportati in Tabella 5.4.

L'agente seleziona una nuova azione ogni 5 frame di simulazione, mantenendola costante nel frattempo (lo stesso meccanismo di frame-skip descritto in Sezione 4.3 per gli agenti a politica continua).

Comando	F_L	F_R
Pass	0.04	0.04
Up	0.08	0.08
Down	0	0
Left	0.0394	0.0406
Right	0.0394	-0.0406

Table 5.4: Spazio di azione discreto dell’agente DQN, come riportato nel paper originale.

Come mostrato nel paper di riferimento [8] DQN non si è rivelato una buona soluzione a questo problema dimostrando forte instabilità legata sia alla rapida variazione del tasso di esplorazione sia alla natura discreta delle azioni disponibili, poco adatta a un compito di controllo intrinsecamente continuo come la stabilizzazione di un quadricottero.

Si noti che l’agente non è incluso nella fase di valutazione. DQN viene quindi qui presentato per completezza, come riferimento storico e motivazione del passaggio ad algoritmi capaci di azioni continue (SAC, A2C, PPO), descritti nella sezione seguente.

5.5 Agenti a politica continua: SAC, A2C, PPO

Gli agenti SAC, A2C, PPO e le loro differenti versioni (nominale, noisy e curriculum) condividono la medesima architettura di interfacciamento con l’ambiente, e differiscono fra loro per l’algoritmo di apprendimento (rispettivamente SAC, A2C e PPO di `stable-baselines3`) e per le condizioni in cui sono stati addestrati.

5.5.1 Osservazione e azione

Tutti questi agenti ricevono il vettore di osservazione normalizzato a 6 componenti (Sezione 4.3.3), eventualmente corrotto da rumore sensoriale (Sezione 4.4.2) per le varianti addestrate in ambiente perturbato.

La policy restituisce due valori continui $(\text{action}_0, \text{action}_1) \in [-1, 1]^2$, convertiti nelle spinte secondo le formule descritte nella sezione 4.3.1

5.5.2 Caricamento dei modelli addestrati

In fase di valutazione, ciascun agente carica i pesi di una rete già addestrata tramite il metodo `.load()` di `stable-baselines3`, senza ulteriore apprendimento online. I file dei modelli, salvati in formato `.zip`, sono riassunti in Tabella 5.5.

Agente	Algoritmo	File del modello
A2C_v1	A2C	a2c_model_v1_1000000_steps.zip
A2C_v1_noise	A2C	a2c_model_v1_noise_4000000_steps.zip
A2C_v1_curriculum	A2C	a2c_model_v1_curriculum_4000000_steps.zip
PPO_v2	PPO	ppo_model_v1_4000000_steps.zip
PPO_v2_noise	PPO	ppo_model_v2_noise_4000000_steps.zip
PPO_v2_curriculum	PPO	ppo_model_v2_curriculum_4000000_steps.zip
SAC_v1	SAC	sac_model_v1_3330000_steps.zip
SAC_v1_noise	SAC	sac_model_v1_noise_3300000_steps.zip
SAC_v1_curriculum	SAC	sac_model_v1_curriculum_3300000_steps.zip

Table 5.5: Modelli caricati dagli agenti RL nell’arena di valutazione.

5.6 Procedure di addestramento

Tutti gli agenti RL sono addestrati con **stable-baselines3** sull’ambiente **droneEnv** descritto in Sezione 4.3 o sull’ambiente **droneEnv_noisy** (ambiente nominale a cui è stato aggiunto del rumore descritto in Sezione 4.4) ,con checkpoint salvati ogni 50 000 passi tramite **CheckpointCallback**.

Di seguito si descrivono in dettaglio le tre procedure di addestramento documentate per l’algoritmo SAC;

A2C e PPO (e le rispettive varianti **nominale**, **noisy** e **curriculum**) seguono una procedura analoga, sostituendo la classe dell’algoritmo (A2C o PPO al posto di SAC) e adattando di conseguenza l’ambiente di training (nominale, perturbato o a curriculum).

5.6.1 Addestramento nominale

Lo script **train_sac.py** addestra il modello SAC_v1 sull’ambiente nominale **env_SAC** (nessun vento, nessun rumore), per 3.3 milioni di timestep complessivi, con i parametri tipici dell’algoritmo SAC: **buffer_size= 50 000**, **batch_size= 256**, coefficiente di entropia regolato automaticamente (**ent_coef="auto"**) e tasso di aggiornamento dei target network $\tau = 0.005$.

5.6.2 Addestramento su ambiente perturbato

Lo script **train_sac_noisy.py** riprende la stessa architettura ma addestra su **env_noisy_SAC** con vento abilitato (**wind_speed_max= 0.04**) e rumore sensoriale abilitato con i valori di default, per un totale di 3.3 milioni di timestep.

5.6.3 Curriculum learning

Lo script **train_sac_curriculum.py** implementa il curriculum learning: lo stesso modello SAC viene addestrato in sequenza su tre ambienti di difficoltà crescente, ciascuno per un terzo del budget totale di 3.3 milioni di timestep (circa 1.1 milioni di passi per fase), continuando l’apprendimento da una fase all’altra tramite

`model.set_env()` anziché ripartire da zero. Le tre fasi, riassunte in Tabella 5.6, aumentano progressivamente l'intensità massima del vento e l'entità del rumore sensoriale.

Fase	Vento (<code>wind_speed_max</code>)	Rumore sensori	Descrizione
0 – Facile	0.01	Disattivato	Ambiente quasi nominale
1 – Media	0.02	Attivo, Σ dimezzata rispetto al default	Difficoltà intermedia
2 – Difficile	0.04	Attivo, Σ di default	Equivalente all'ambiente “noisy” completo

Table 5.6: Le tre fasi del curriculum learning, a difficoltà progressivamente crescente.

L'idea alla base di questo approccio è che esporre l'agente a perturbazioni lievi nelle prime fasi di apprendimento gli permetta di acquisire prima una politica di base stabile (raggiungere i target senza schiantarsi), per poi affinarla progressivamente alla presenza di disturbi più severi, anziché dover apprendere simultaneamente sia il compito di base sia la robustezza al rumore fin dal primo timestep.

Capitolo 6

Esperimenti e risultati

In questo capitolo vengono presentati i risultati ottenuti dall’addestramento e dalla valutazione degli agenti descritti nel Capitolo 5. Prima di entrare nel merito, è opportuno chiarire il protocollo sperimentale e le metriche utilizzate.

6.1 Protocollo di valutazione e metriche

6.1.1 Metrica di valutazione

Il framework sviluppato prevede due modalità di valutazione complementari, ciascuna con uno scopo distinto.

La prima modalità è il **reward cumulato di addestramento**, definito in Sezione 4.3.4 per gli ambienti `droneEnv` e `droneEnv_noisy`. Questo segnale viene calcolato frame per frame durante il training ed è utilizzato dagli algoritmi di Reinforcement Learning come funzione obiettivo da massimizzare. Si tratta di una grandezza *interna* al processo di ottimizzazione: è disponibile esclusivamente per gli agenti addestrati con `stable-baselines3` (A2C, PPO e SAC) e non è direttamente comparabile con i controllori PID o Human, che non ottimizzano esplicitamente alcuna funzione obiettivo. Le curve di training mostrate nelle sezioni successive derivano da questo segnale e permettono di analizzare la velocità di convergenza, la stabilità e la presenza di fenomeni di regressione durante l’apprendimento.

La seconda modalità comprende le **metriche di performance task-level**, utilizzate per confrontare *tutti* gli agenti su un piano comune. La metrica principale è il **numero medio di palloncini raccolti per episodio**, misurato eseguendo ogni agente addestrato sull’ambiente `droneEnv` o `droneEnv_noisy` descritto in Sezione 4.3. Ogni episodio ha una durata massima di 20 s; ogni volta che un target viene raggiunto ($d < 50$ px) ne viene generato automaticamente uno nuovo in posizione casuale, quindi il conteggio finale riflette congiuntamente la velocità di navigazione e la capacità di evitare i crash.

La metrica secondaria è la **percentuale di crash**: un episodio termina anticipatamente quando la distanza dal target supera 1000 px, corrispondente alla penalità di -1000 nella funzione di reward. Un tasso di crash elevato indica una

policy strutturalmente instabile, indipendentemente dal reward di training.

Ciascuna configurazione è stata valutata su **1000 episodi indipendenti** per ottenere stime statisticamente stabili.

6.1.2 Metriche derivate

Per rendere il confronto tra algoritmi più rigoroso, oltre al valore di plateau del reward vengono considerate:

- **velocità di convergenza:** numero di timestep necessari per raggiungere (e mantenere) il plateau di reward;
- **stabilità:** presenza/assenza di crolli improvvisi del reward (riconducibili a crash ripetuti) nelle fasi avanzate del training, osservata qualitativamente sulle curve;
- **gap di robustezza:** differenza tra il reward di plateau in ambiente nominale e in ambiente perturbato, come indicatore di quanto la policy degradi in presenza di vento e rumore sensoriale.

6.1.3 Configurazioni valutate

Per ciascun algoritmo (A2C, PPO, SAC) sono stati valutati tre modelli addestrati in condizioni diverse:

- **nominale:** addestrato su `droneEnv` senza disturbi;
- **noisy:** addestrato direttamente su `droneEnv_noisy` (vento + rumore sensoriale);
- **curriculum:** addestrato progressivamente su tre fasi di difficoltà crescente (Sezione 5.6.3).

Ciascun modello è stato valutato su **entrambi** gli ambienti (nominale e perturbato), per misurare sia le prestazioni nel dominio di addestramento sia la capacità di trasferimento al dominio opposto. Questo schema 3×2 per algoritmo permette di separare l'effetto della strategia di addestramento dall'effetto dell'ambiente di test.

6.2 Hyperparameter Tuning

Prima del confronto finale, è stata condotta una fase di ricerca degli iperparametri, con strategie diverse a seconda del costo computazionale di ciascun algoritmo.

Per **A2C** e **PPO**, grazie alla loro efficienza computazionale, è stato possibile eseguire una *grid search* automatizzata.

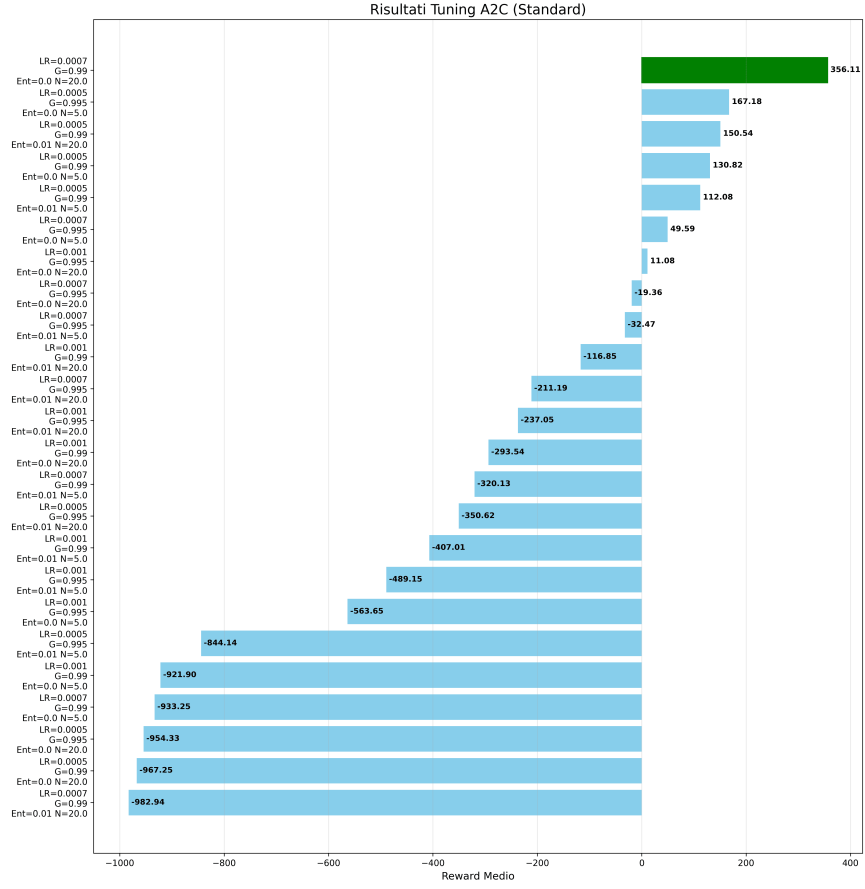


Figure 6.1: Risultati della Grid Search per A2C. La configurazione migliore (evidenziata) ha ottenuto un reward medio di 356.11 durante la fase di tuning, eseguita con budget ridotto rispetto al training finale.

Come mostrato in Figura 6.1, A2C si è rivelato sensibile alla scelta degli iperparametri: la configurazione migliore ($LR = 7 \cdot 10^{-4}$, $N_{\text{steps}} = 20$) si distacca nettamente dalle altre, molte delle quali producono reward negativi. Questo è un primo indizio della minore robustezza intrinseca dell'algoritmo, confermato poi dai risultati di valutazione (Sezione 6.6).

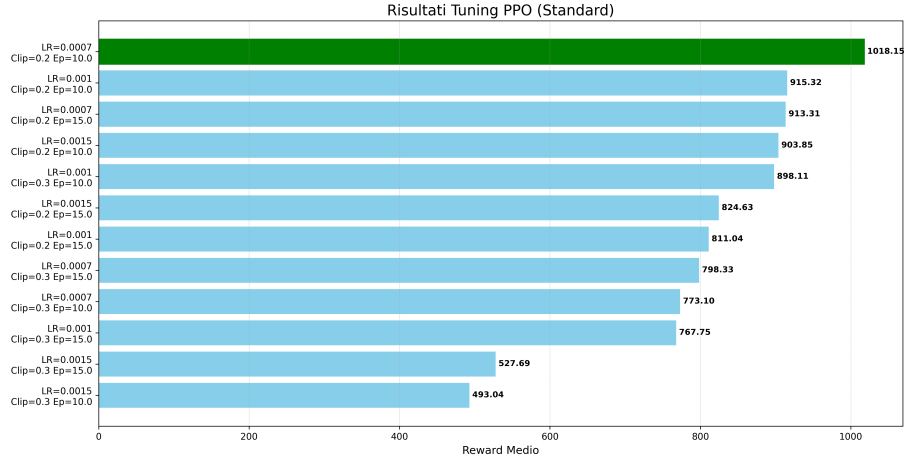


Figure 6.2: Risultati della Grid Search per PPO. La maggior parte delle configurazioni testate converge a reward positivi ed elevati (> 700), con il massimo raggiunto da $LR = 7 \cdot 10^{-4}$, $Clip = 0.2$ (> 1000).

PPO mostra una robustezza nettamente superiore alla scelta degli iperparametri: quasi tutte le combinazioni testate raggiungono reward elevati, segno di un algoritmo meno sensibile al tuning rispetto ad A2C.

Per **SAC**, il maggiore costo computazionale ha reso impraticabile una grid search della stessa ampiezza. È stato quindi adottato un **tuning manuale**, basato su un numero ridotto di configurazioni derivate dalla letteratura di riferimento. Questa asimmetria metodologica va tenuta presente: il fatto che SAC risulti comunque l'algoritmo più performante nonostante un tuning meno estensivo ne sottolinea ulteriormente la robustezza intrinseca.

6.3 Risultati in ambiente nominale

In questa sezione vengono analizzate le curve di training degli agenti addestrati in ambiente nominale, privo di vento e rumore sensoriale. Per ciascun algoritmo viene descritto separatamente il comportamento osservato.

6.3.1 A2C in ambiente nominale

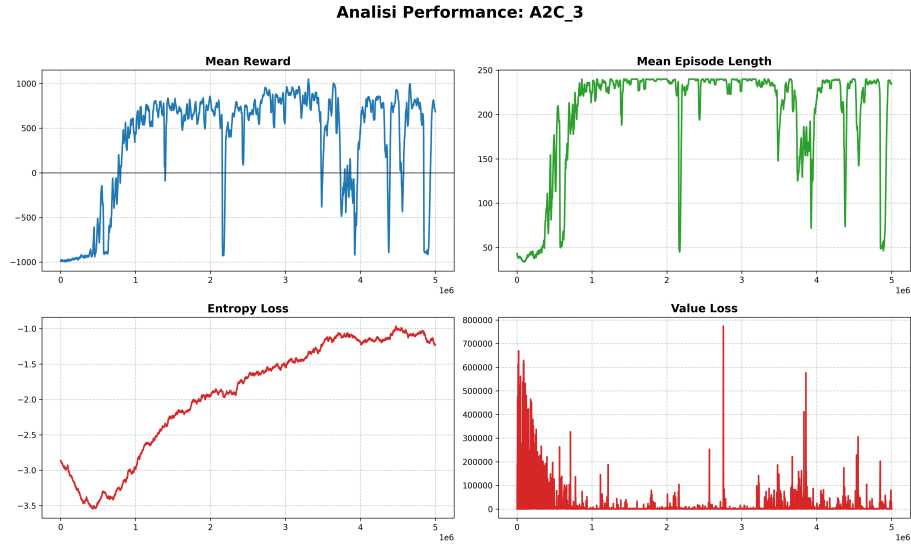


Figure 6.3: A2C in ambiente nominale. Pur raggiungendo picchi di reward fino a 1000, la curva mostra crolli ricorrenti verso valori fortemente negativi per l'intero arco del training (a 2.2M, 3.5M, 4.0M e 4.5M step), con episodi di crash che si ripetono anche nelle fasi molto avanzate. La *value loss* non converge stabilmente, presentando picchi ricorrenti fino a 800 000 anche oltre i 2.5M step.

A2C raggiunge picchi di reward comparabili a quelli degli altri algoritmi, ma non riesce a mantenerli nel tempo. I crolli periodici verso reward fortemente negativi, distribuiti lungo l'intero training, suggeriscono un fenomeno di instabilità strutturale della policy: il modello apprende una strategia efficace, poi la perde e deve riacquisirla ciclicamente. Questo comportamento è plausibilmente legato all'assenza, in A2C, di meccanismi di *trust region* come il clipping di PPO, che limitano la portata degli aggiornamenti della policy. La *value loss* rimane alta e irregolare per tutta la durata del training, con picchi impulsivi che si ripresentano a distanza anche nella seconda metà del training.

6.3.2 PPO in ambiente nominale

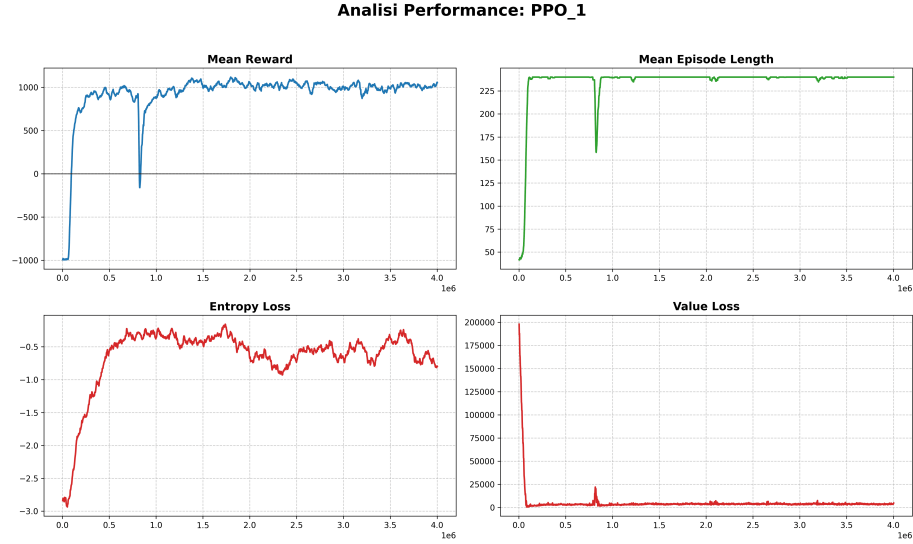


Figure 6.4: PPO in ambiente nominale. La curva converge in modo relativamente rapido, raggiungendo un plateau attorno a 1000 entro i primi 500 000 step. Si nota un episodio di crollo isolato attorno a 800 000 step, da cui il modello si riprende prontamente. La *value loss* si riduce stabilmente dopo la fase iniziale, indicando una buona convergenza del Critic.

PPO mostra un comportamento qualitativamente superiore ad A2C. La convergenza avviene entro i primi 500 000 step e, nonostante un singolo episodio di regressione attorno a 800 000 step da cui il modello si riprende rapidamente, il plateau viene mantenuto senza crisi sistematiche. La *value loss* tende a stabilizzarsi dopo la fase iniziale, confermando che il meccanismo di clipping della funzione obiettivo svolge un ruolo protettivo contro gli aggiornamenti destabilizzanti. Rispetto ad A2C, la curva di reward di PPO è notevolmente più regolare nella parte centrale e finale del training.

6.3.3 SAC in ambiente nominale

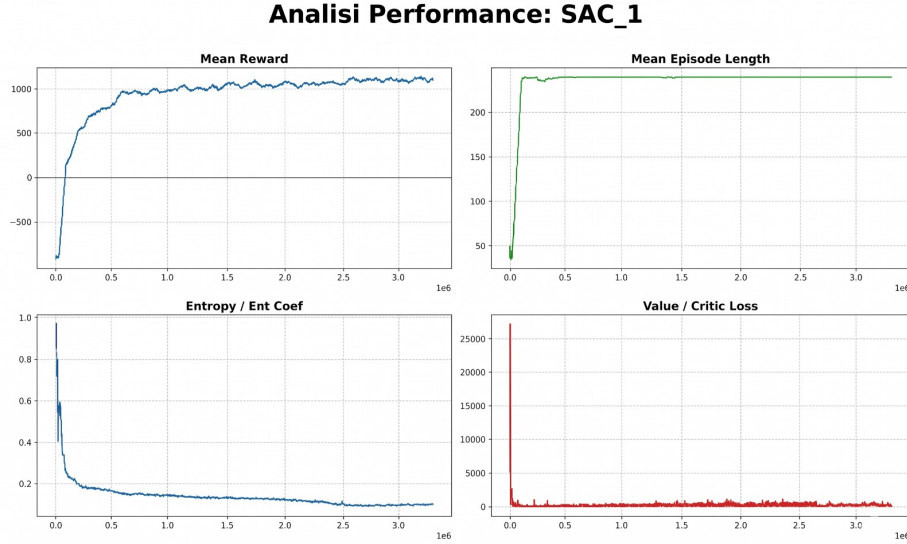


Figure 6.5: SAC in ambiente nominale. La curva converge con grande rapidità: il reward supera 500 già entro i primi 50 000 step e raggiunge un plateau stabile oltre 1000 attorno a 300 000 step, che viene mantenuto per l'intera durata del training senza crolli. L'entropia si azzerava rapidamente da 1.0 a circa 0.11, e la *value loss* presenta un unico picco iniziale molto elevato (attorno a 27 000) che si azzerava pressoché immediatamente.

SAC si distingue nettamente dagli altri due algoritmi in ambiente nominale. La convergenza è la più rapida osservata in tutta la sperimentazione: il plateau di reward oltre 1000 viene raggiunto già attorno a 300 000 step e mantenuto in modo estremamente stabile per tutti i restanti 3M di step, senza alcun crollo. La *value loss* mostra un comportamento ideale: un unico picco iniziale di grande ampiezza legato alla fase di esplorazione, seguito da un crollo rapido a valori prossimi a zero. L'entropia si riduce progressivamente indicando una policy che si specializza in modo ordinato senza perdere mai il controllo della stabilità. Rispetto a PPO e soprattutto ad A2C, SAC dimostra una capacità di mantenimento del plateau nettamente superiore, frutto della combinazione tra il replay buffer off-policy e la regolarizzazione entropica.

6.4 Robustezza in ambiente perturbato

In questa sezione vengono confrontate le curve di training degli agenti addestrati direttamente sull'ambiente perturbato (vento con $\text{speed}_{\max} = 0.04$ e rumore sensoriale di default), senza curriculum progressivo.

6.4.1 A2C in ambiente perturbato

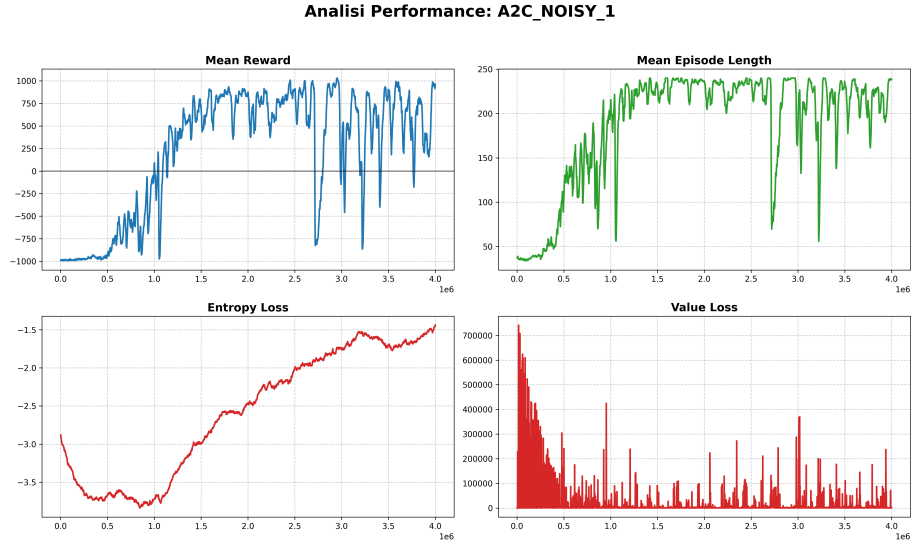


Figure 6.6: A2C in ambiente Noisy (training diretto). Il reward cresce lentamente fino a circa 1.5M step e poi continua ad oscillare con varianza molto elevata tra -750 e $+1000$ senza mai stabilizzarsi. La *value loss* rimane irregolare per tutto il training con picchi ripetuti anche nella seconda metà, e la convergenza stabile non viene raggiunta.

Per A2C, il rumore aggiuntivo amplifica una difficoltà strutturale preesistente: la convergenza è ancora più lenta (il reward positivo si mantiene stabilmente solo dopo 1.5M step) e la varianza del reward non si riduce nelle fasi avanzate.

6.4.2 PPO in ambiente perturbato

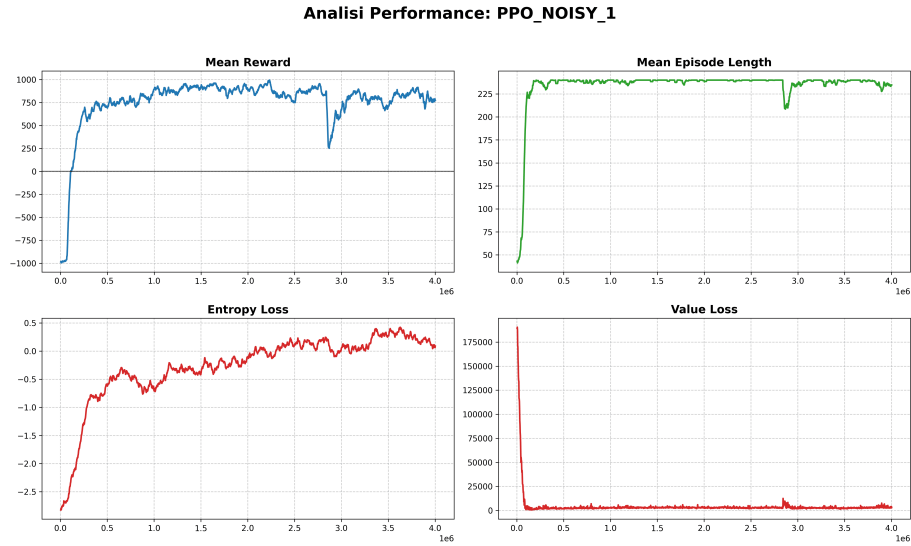


Figure 6.7: PPO in ambiente Noisy (training diretto). La policy si stabilizza su un buon reward (attorno a 800–900) entro i primi 500 000 step, ma mostra un crollo marcato attorno a 2.9M step da cui si riprende solo parzialmente. La convergenza è più lenta e meno stabile rispetto al caso nominale.

PPO mostra un degrado contenuto rispetto al caso nominale, con un plateau inferiore e una maggiore instabilità nella seconda metà del training. Il crollo attorno a 2.9M step è più pronunciato di quello osservato nel caso nominale, suggerendo che le perturbazioni ambientali rendano più difficile mantenere la policy una volta appresa.

6.4.3 SAC in ambiente perturbato

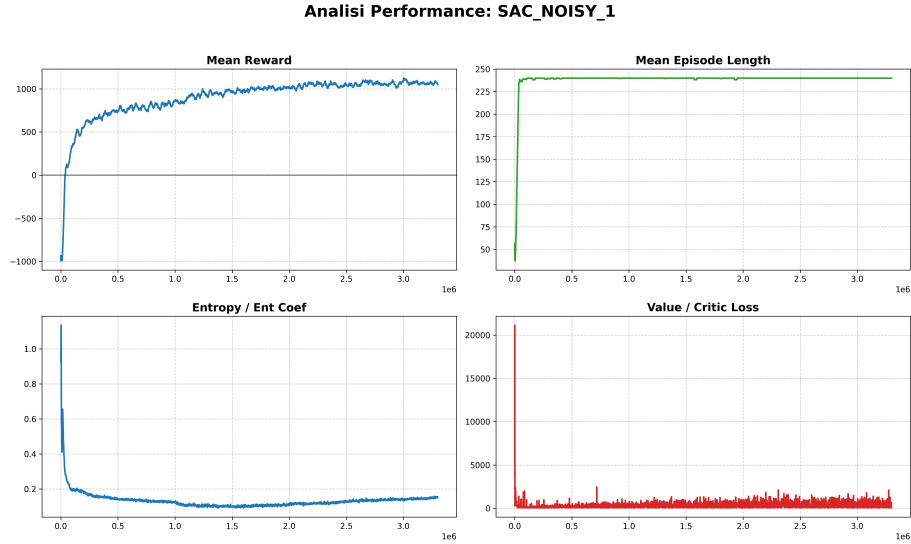


Figure 6.8: SAC in ambiente Noisy (training diretto). La convergenza viene raggiunta con notevole rapidità: il reward supera 500 già entro i primi 50 000 step e si stabilizza sopra 1000 attorno a 500 000 step. La *value loss* presenta un unico picco iniziale molto elevato (attorno a 21 000) che si azzerava rapidamente, e l'entropia si assesta su un valore stabile attorno a 0.16, indicando un bilanciamento efficace tra sfruttamento ed esplorazione.

SAC raggiunge la convergenza in un numero di step nettamente inferiore rispetto a PPO e A2C. La componente di massimizzazione dell'entropia favorisce l'esplorazione di micro-correzioni utili a compensare un disturbo stocastico come il vento, contribuendo alla rapida stabilizzazione della policy. Il plateau di reward supera 1000 e viene mantenuto con varianza molto ridotta per tutta la durata del training.

6.5 Impatto del Curriculum Learning

Il Curriculum Learning (Sezione 5.6.3) è stato valutato confrontando, per ciascun algoritmo, il training diretto in ambiente perturbato con il training progressivo a tre fasi.

6.5.1 A2C: il curriculum non è sufficiente

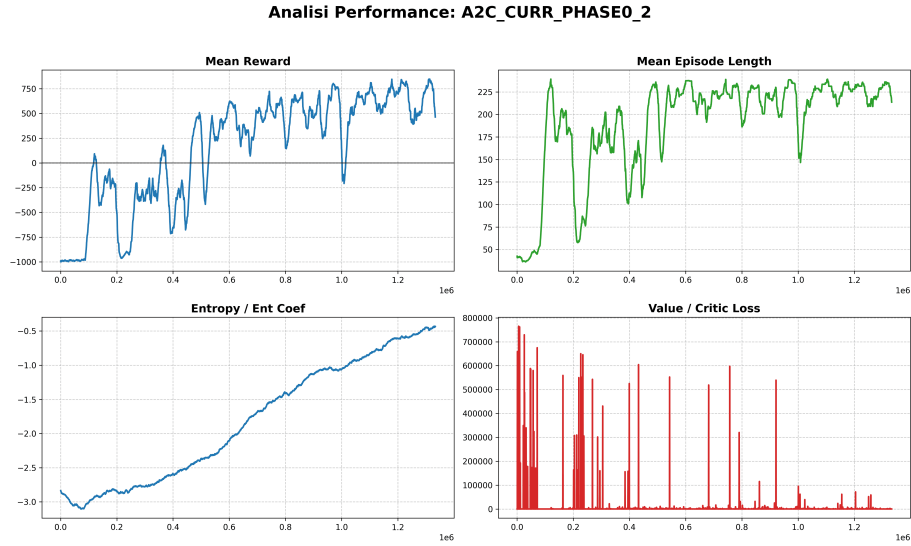


Figure 6.9: Curriculum A2C, Fase 0 (Easy). Il reward parte da -1000 e sale lentamente fino a valori attorno a $750-800$, ma con oscillazioni marcate e crolli periodici, soprattutto nella prima metà del training. La *value loss* mostra picchi impulsivi molto frequenti nella fase iniziale che si diradano solo parzialmente.

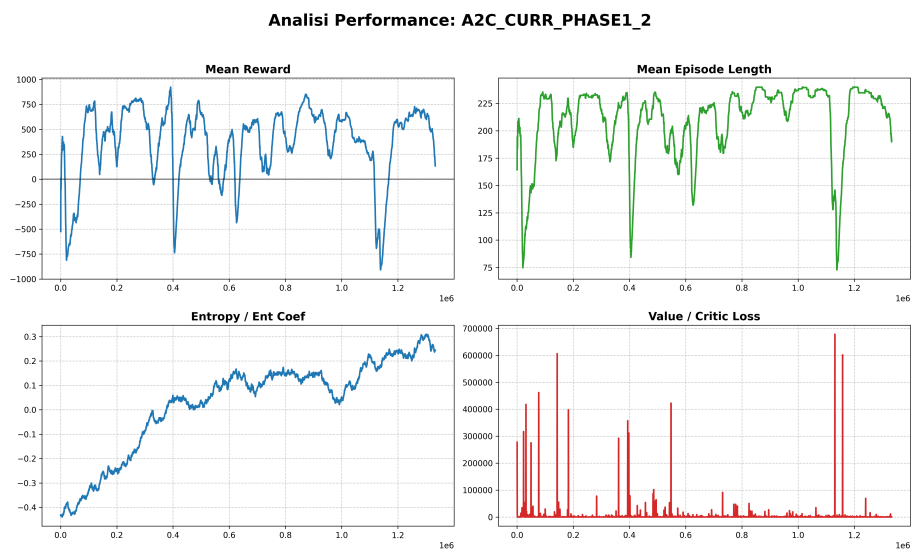


Figure 6.10: Curriculum A2C, Fase 1 (Medium). Il reward oscilla continuamente tra -750 e $+900$ senza mai raggiungere un plateau stabile. Si osservano crolli profondi distribuiti lungo tutto il training. La *value loss* presenta picchi elevati e ricorrenti, indicando che il Critic fatica a stabilizzarsi anche con perturbazioni intermedie.

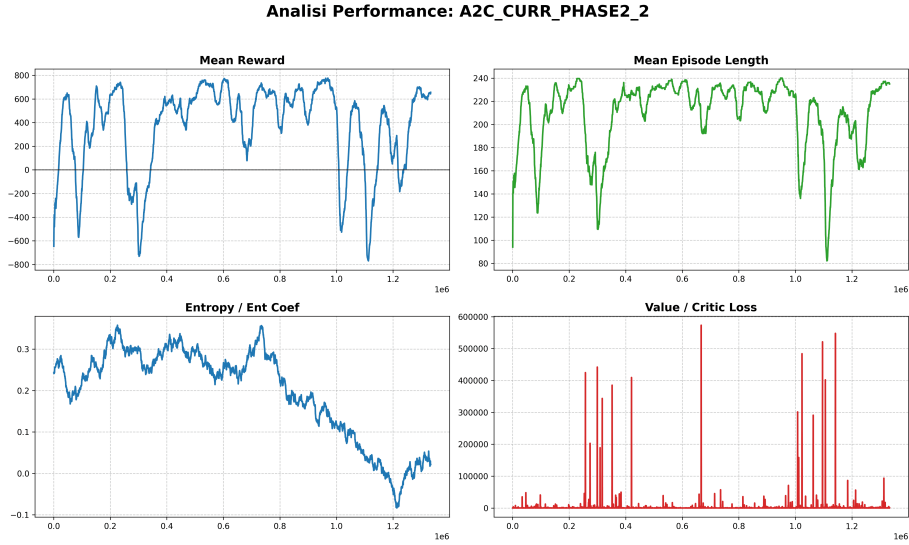


Figure 6.11: Curriculum A2C, Fase 2 (Hard). L’instabilità osservata nelle fasi precedenti non solo persiste ma si intensifica: il reward oscilla tra -800 e $+750$ senza tendenza positiva, con crolli profondi anche nella seconda metà del training. La *value loss* mostra i picchi più elevati dell’intera sequenza curriculum (fino a 600 000), suggerendo che il modello fatica a generalizzare quanto appreso nelle fasi facili.

Nelle Fasi 0 e 1 l’agente apprende in modo progressivo ma con oscillazioni marcate sin dall’inizio. Nella Fase 2, tuttavia, il reward non mostra alcun miglioramento rispetto alla fase precedente, e le oscillazioni si mantengono su ampiezza simile.

Questo suggerisce che il limite di A2C non sia tanto la difficoltà di esplorazione iniziale, che il curriculum mitiga parzialmente, quanto una limitazione strutturale dell’algoritmo (assenza di replay buffer, aggiornamenti on-policy ad alta varianza) nel gestire ambienti fortemente stocastici. Il warm start della Fase 2 non si traduce in una policy più stabile, come invece avviene per PPO.

6.5.2 PPO: riduzione della varianza

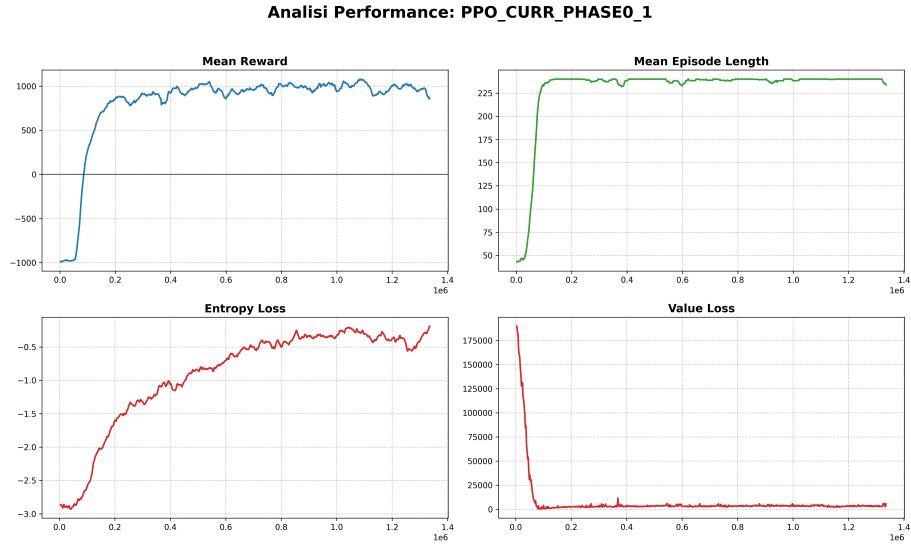


Figure 6.12: Curriculum PPO, Fase 0 (Easy). La policy converge rapidamente entro i primi 100 000–200 000 step, raggiungendo un plateau stabile attorno a 950–1000. La *value loss* si azzerava rapidamente dopo il picco iniziale.

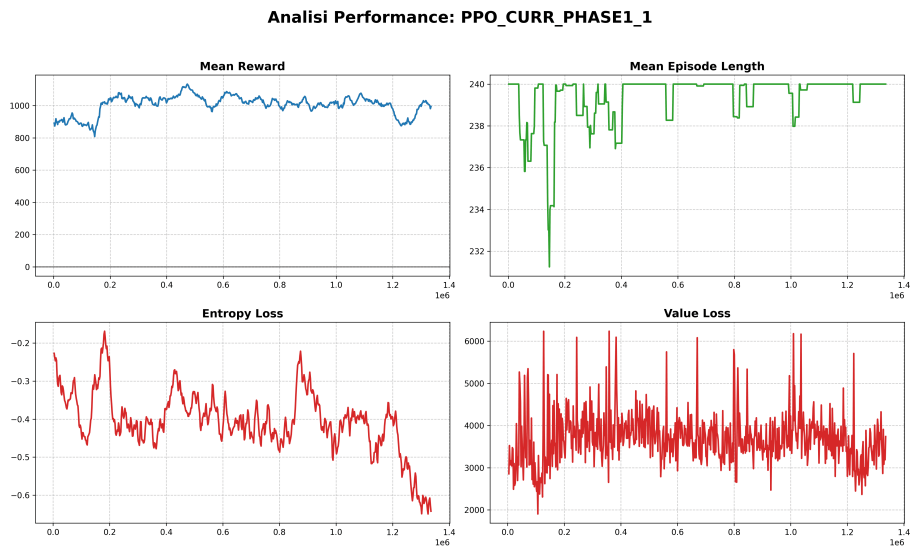


Figure 6.13: Curriculum PPO, Fase 1 (Medium). Grazie al warm start dalla Fase 0, il reward parte già da valori elevati (≈ 900) e si mantiene attorno a 1000 per tutto il training. Le oscillazioni sono presenti ma contenute, e non si osservano crolli profondi come nel training diretto su ambiente noisy.

Analisi Performance: PPO_CURR_PHASE2_1

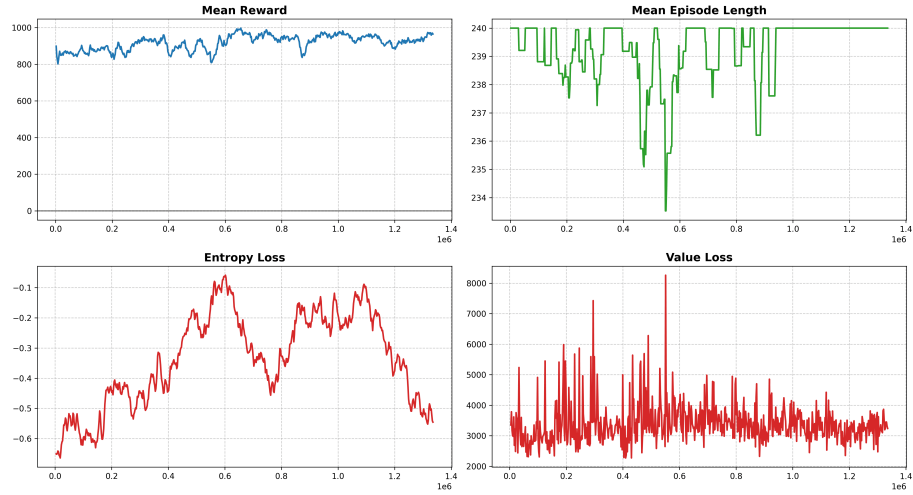


Figure 6.14: Curriculum PPO, Fase 2 (Hard). Il reward si mantiene stabilmente vicino al massimo (850–1000) con varianza visibilmente inferiore rispetto al training diretto (Figura 6.7). Non si osserva il crollo profondo che caratterizza la variante noisy diretta. La *value loss* rimane su livelli contenuti per tutta la fase.

A differenza di A2C, PPO beneficia chiaramente del curriculum in tutte e tre le fasi. Già dalla Fase 0 la convergenza è più rapida e stabile rispetto al training nominale diretto; nella Fase 1 il warm start si traduce in un avvio del training già a reward elevato; nella Fase 2 la curva è quasi piatta attorno al valore massimo, in netto contrasto con il crollo a 2.9M step osservato nel training diretto su ambiente noisy. L'interpretazione più diretta è che la policy, già stabilizzata nelle fasi precedenti, debba operare solo piccoli adattamenti incrementali nella fase difficile.

6.5.3 SAC: curriculum poco rilevante

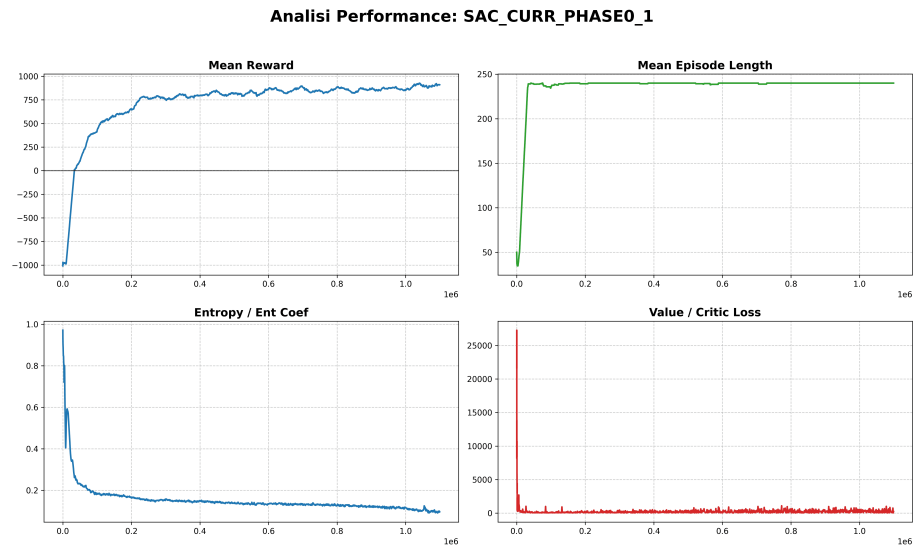


Figure 6.15: Curriculum SAC, Fase 0 (Easy). SAC converge già in questa prima fase entro 200 000 step, raggiungendo un plateau attorno a 850. L'entropia si riduce rapidamente da 1.0 a circa 0.13, e la *value loss* presenta un unico picco iniziale seguito da un crollo stabile.

Analisi Performance: SAC_CURR_PHASE1_1

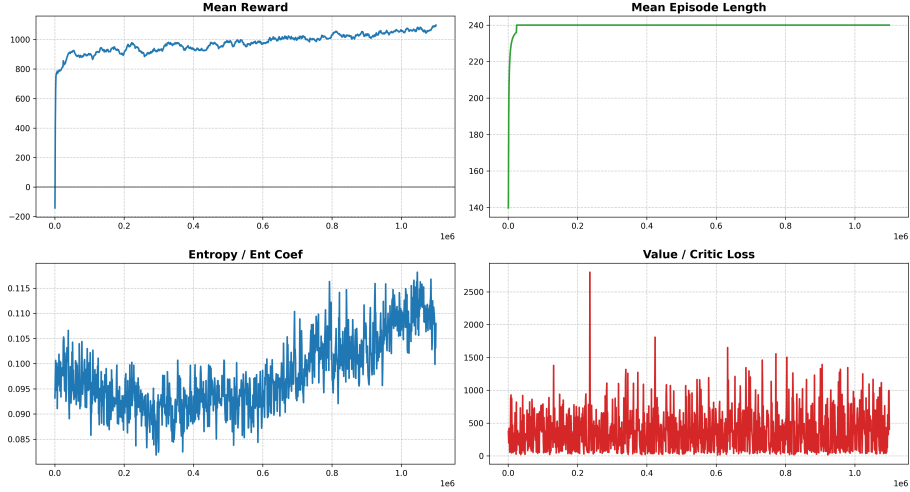


Figure 6.16: Curriculum SAC, Fase 1 (Medium). Partendo dal warm start della Fase 0, il reward raggiunge il plateau massimo (> 1000) quasi immediatamente. L'entropia sale leggermente rispetto alla Fase 0, indicando una ricalibrazione esplorativa per adattarsi alle nuove perturbazioni.

Analisi Performance: SAC_CURR_PHASE2_1

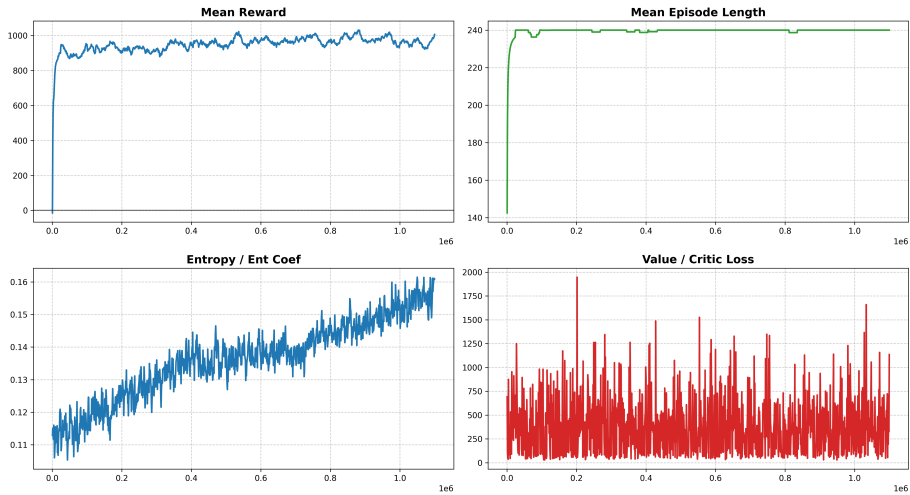


Figure 6.17: Curriculum SAC, Fase 2 (Hard). Anche nella fase più difficile, SAC raggiunge il reward massimo in tempi brevi (entro 30 000–50 000 step). Il plateau attorno a 950–1000 viene mantenuto per tutta la fase con varianza ridotta, confermando l'elevata *sample efficiency* dell'algoritmo.

Per SAC il curriculum non produce un miglioramento apprezzabile rispetto al training diretto: in entrambi i casi l'algoritmo satura rapidamente il reward disponibile. In Fase 0 SAC converge già a un plateau elevato partendo da zero; nelle Fasi 1 e 2 il warm start viene sfruttato ma non è determinante, dato che SAC convergerebbe comunque rapidamente anche partendo ex novo. Questo conferma l'elevata *sample efficiency* già osservata in Sezione 6.4, che rende il curriculum una precauzione non necessaria per questo algoritmo.

6.6 Confronto quantitativo: palloncini raccolti

Le sezioni precedenti analizzano le curve di training (reward cumulato), che sono una metrica interna all'ottimizzazione RL e non confrontabile direttamente con il controllore PID. In questa sezione si utilizza la metrica task-level descritta in Sezione 6.1: il numero medio di palloncini raccolti per episodio su 1000 episodi indipendenti, sia in ambiente nominale sia in ambiente perturbato.

6.6.1 Risultati in ambiente nominale

La Tabella 6.1 riporta i risultati di tutti i modelli addestrati, valutati sull'**ambiente nominale** (`droneEnv`), indipendentemente dal dominio in cui sono stati addestrati. Per il controllore Human vengono riportati i valori stimati sulla base di sessioni di valutazione qualitative nell'arena BalloonEnv; i valori sono da considerarsi indicativi.

Algoritmo	Strategia di training	Palloncini/ep	Crash%
Human	Controllo manuale	$\approx 4-6$	elevata
PID	Controllo classico	11.02	0.0%
A2C	Nominale	7.43	9.0%
A2C	Noisy	10.31	0.0%
A2C	Curriculum	8.72	1.0%
PPO	Nominale	10.54	0.0%
PPO	Noisy	8.96	2.4%
PPO	Curriculum	10.87	0.0%
SAC	Nominale	9.33	0.0%
SAC	Noisy	11.47	0.0%
SAC	Curriculum	11.33	0.0%

Table 6.1: Palloncini medi per episodio e tasso di crash in ambiente **nominale**. In grassetto i valori più alti tra gli agenti RL.

In ambiente nominale, **SAC** ottiene i punteggi più alti (11.47 per la variante noisy, 11.33 per la curriculum), seguito da PPO curriculum (10.87).

Il controllore PID si colloca in una fascia comparabile ai migliori agenti, con un punteggio calcolato attorno a 11 palloncini/ep e un tasso di crash praticamente nullo in assenza di perturbazioni: un risultato notevole per un sistema basato su regole fisse e non su apprendimento.

L'agente umano, come atteso, mostra le prestazioni più basse in termini di palloncini raccolti, con un tasso di crash elevato dovuto alla difficoltà di coordinare manualmente i due propulsori.

Degno di nota è il comportamento di **A2C nominale**, che raggiunge il punteggio più basso tra i modelli A2C (7.43) con un tasso di crash del 9.0%: la policy appresa in ambiente pulito si rivela quindi meno robusta di quanto suggerito dal reward di training, a conferma della tendenza all'instabilità già osservata nelle curve di Figura 6.3.

Al contrario, il modello **A2C noisy**, addestrato in ambiente perturbato, ottiene 10.31 palloncini/ep con 0% di crash anche in ambiente nominale, suggerendo che l'esposizione alle perturbazioni durante il training abbia avuto un effetto regolarizzante sulla policy.

6.6.2 Risultati in ambiente perturbato

La Tabella 6.2 riporta gli stessi modelli valutati sull'**ambiente perturbato** (`droneEnv.noisy`), con vento e rumore sensoriale attivi.

Algoritmo	Strategia di training	Palloncini/ep	Crash%
Human	Controllo manuale	$\approx 2-3$	molto elevata
PID	Controllo classico	5.32	12.8%
A2C	Nominale	6.80	11.2%
A2C	Noisy	9.72	2.4%
A2C	Curriculum	8.11	3.4%
PPO	Nominale	8.91	0.6%
PPO	Noisy	8.24	8.8%
PPO	Curriculum	9.55	0.2%
SAC	Nominale	6.24	0.4%
SAC	Noisy	10.56	0.0%
SAC	Curriculum	10.10	0.0%

Table 6.2: Palloncini medi per episodio e tasso di crash in ambiente **perturbato** (1000 episodi per configurazione RL, valori indicativi per PID e Human). In grassetto i valori più alti tra gli agenti RL.

In ambiente perturbato emergono le differenze più significative tra i controllori. Il **PID** subisce un degrado drastico: il tasso di crash sale al 12.8% e i palloncini raccolti per episodio scendono a 5.32, circa la metà rispetto all'ambiente nominale. Questo comportamento riflette i limiti strutturali del PID descritti nel Capitolo 2.5: l'azione derivativa amplifica il rumore sensoriale, mentre l'azione integrale non riesce a compensare disturbi che cambiano direzione nel tempo come il vento variabile. Il risultato è un'oscillazione persistente attorno al target e un aumento significativo dei crash. L'agente umano risulta ancora più penalizzato dal rumore sensoriale, poiché le osservazioni corrotte rendono il controllo manuale particolarmente difficile.

SAC noisy si conferma il modello più performante (10.56 palloncini/ep, 0% crash).

Per **PPO**, il modello noisy peggiora in ambiente perturbato rispetto al nominale (8.24 vs 8.91) con un tasso di crash dell'8.8%: un risultato apparentemente controintuitivo, ma spiegabile con il fatto che l'addestramento diretto su perturbazioni intense, senza la progressività del curriculum, può portare a una policy eccessivamente conservativa che rinuncia a manovre rischiose ma necessarie.

Il modello **PPO curriculum** risolve questo problema, raggiungendo 9.55 palloncini/ep con appena lo 0.2% di crash, il miglior risultato per PPO in assoluto. Questo conferma l'osservazione qualitativa della Sezione 6.5: PPO beneficia significativamente del curriculum.

6.7 Analisi per algoritmo

6.7.1 A2C

A2C mostra un comportamento incoerente tra le varianti. Il modello nominale è il più debole in assoluto: 7.43 palloncini/ep con un tasso di crash del 9.0% in ambiente pulito, il che è sorprendente per un modello addestrato precisamente in quell'ambiente.

Questo risultato si spiega con l'instabilità della curva di training osservata in Figura 6.3: i crolli ripetuti del reward nelle fasi avanzate del training, visibili attorno a 2.2M, 3.5M, 4.0M e 4.5M step, indicano che il modello non converge a un minimo stabile ma oscilla continuamente. Il checkpoint salvato alla fine del training potrebbe non corrispondere a un picco di performance.

Il modello **A2C noisy** è invece il migliore della famiglia A2C in entrambi gli ambienti, superando persino il curriculum. Questo suggerisce che, per A2C, il training diretto su ambiente perturbato sia più efficace del curriculum: le perturbazioni costanti fungono da regolarizzatore implicito, riducendo le oscillazioni della policy rispetto a quanto osservato nell'ambiente nominale.

In ambiente perturbato, il modello nominale A2C è l'unico a superare l'11% di crash, il valore più alto dell'intera sperimentazione RL, a conferma della scarsa generalizzazione tra domini. Il curriculum migliora A2C rispetto alla variante nominale, ma non raggiunge le prestazioni del training diretto su noisy, a riprova che la progressività non compensa le limitazioni strutturali dell'algoritmo in ambiente stocastico.

6.7.2 PPO

PPO mostra un comportamento più coerente e prevedibile. In ambiente nominale, il modello nominale e quello curriculum ottengono risultati comparabili (10.54 vs 10.87), entrambi con 0% di crash. Il modello noisy è invece meno performante (8.96) con crash al 2.4%: l'addestramento diretto su perturbazioni

intense ha reso la policy più cauta, riducendo i palloncini raccolti ma non eliminando del tutto i crash in ambiente perturbato.

In ambiente perturbato, il modello curriculum si afferma come la scelta migliore per PPO (9.55, 0.2% crash), confermando quanto osservato qualitativamente in Sezione 6.5. Questo risultato è coerente con le curve di training delle Fasi 0, 1 e 2: la progressività permette a PPO di costruire una policy stabile partendo da condizioni semplici, adattandola gradualmente alle perturbazioni senza le instabilità del training diretto. Il modello noisy invece peggiora vistosamente in ambiente perturbato (8.8% crash), suggerendo che il training diretto su ambiente difficile introduca una certa fragilità nella policy, un comportamento opposto a quanto osservato per SAC.

PPO curriculum rappresenta quindi la scelta più equilibrata per questo algoritmo: ottiene il punteggio più alto in ambiente nominale e il secondo più alto in ambiente perturbato, con tassi di crash trascurabili in entrambi i casi.

6.7.3 SAC

SAC presenta la dinamica più interessante. In ambiente nominale, il modello addestrato su **droneEnv** puro raggiunge 9.33 palloncini/ep con 0% di crash, un risultato già competitivo. Tuttavia, le varianti noisy e curriculum lo superano nettamente (11.47 e 11.33 rispettivamente), suggerendo che l'esposizione alle perturbazioni durante il training abbia un effetto regolarizzante positivo anche quando il deployment avviene in ambiente pulito.

In ambiente perturbato, il modello **SAC nominale** subisce il calo più marcato tra tutti i modelli RL: da 9.33 scende a 6.24 palloncini/ep (-33%), il gap di robustezza più elevato dell'intera sperimentazione (3.09). Nonostante il tasso di crash rimanga basso (0.4%), la navigazione risulta inefficace: il modello riesce a sopravvivere nel dominio perturbato ma non a compensare attivamente i disturbi, mancando delle strategie sviluppate solo con l'esposizione al rumore e al vento durante il training. Questo comportamento è coerente con la natura dell'algoritmo: la massimizzazione dell'entropia spinge all'esplorazione ottimale in condizioni note, ma senza disturbi durante il training non genera meccanismi di compensazione attiva.

Il modello **SAC noisy** si conferma il più performante in assoluto (10.56, 0% crash in ambiente perturbato), mentre il curriculum è efficace ma non necessario: raggiunge 10.10 in ambiente perturbato, leggermente al di sotto. Considerando che il curriculum introduce un costo di training aggiuntivo senza benefici misurabili per SAC, il training diretto su ambiente perturbato risulta la strategia preferibile per questo algoritmo.

6.7.4 Confronto PID vs RL

Il controllore PID rappresenta una baseline competitiva in ambiente nominale, con prestazioni comparabili ai migliori modelli PPO e A2C (11.02 palloncini/ep, crash 0%). Tuttavia, in ambiente perturbato le sue prestazioni degradano in modo molto più marcato rispetto agli agenti RL, con un gap stimato di circa

5 palloncini/ep e un tasso di crash che sale al 12.8%. Questo comportamento riflette una caratteristica fondamentale del PID: la sua sintonizzazione è ottimizzata per le condizioni nominali e non può adattarsi automaticamente a perturbazioni che cambiano in modo stocastico. In particolare, il rumore sensoriale degrada l'azione derivativa (che amplifica il rumore sulla derivata dell'errore) e il vento non stazionario sfida l'azione integrale, che accumula errore in direzioni che cambiano nel tempo.

Gli agenti RL, avendo appreso la policy dall'esperienza in ambiente perturbato, sviluppano strategie di compensazione implicite che il PID non può acquisire senza una nuova fase di tuning.

Capitolo 7

Conclusioni e sviluppi futuri

In questo capitolo vengono sintetizzati i risultati principali emersi dalla sperimentazione, si discutono i limiti metodologici del lavoro svolto e si propongono possibili direzioni di sviluppo future.

7.1 Sintesi dei risultati

La Tabella 7.1 riassume le prestazioni del **modello migliore per ciascun algoritmo** nei due ambienti di test, selezionando la variante di training che ottiene il punteggio più alto in ambiente perturbato (il dominio più critico).

Algoritmo	Miglior variante	Palloncini/ep		Crash%	
		Nominale	Noisy	Nominale	Noisy
Human	Manuale	≈ 5	≈ 2.5	elevata	molto elevata
PID	Classico	11.02	5.32	0%	12.8%
A2C	Noisy	10.31	9.72	0.0%	2.4%
PPO	Curriculum	10.87	9.55	0.0%	0.2%
SAC	Noisy	11.47	10.56	0.0%	0.0%

Table 7.1: Confronto tra i migliori modelli per ciascun controllore nei due ambienti di valutazione (1000 episodi per RL). I valori DQN, Human e PID sono riferimenti qualitativi/indicativi e non direttamente comparabili su base metrica sistematica.

I risultati portano a cinque conclusioni principali:

- **SAC è l'algoritmo più performante in assoluto**, sia in ambiente nominale (11.47 palloncini/ep) sia in quello perturbato (10.56), con tasso di crash nullo in entrambi i casi quando addestrato su ambiente perturbato. La sua elevata *sample efficiency*, garantita dal replay buffer off-policy, e la massimizzazione dell'entropia, che mantiene un'esplorazione robusta nelle fasi avanzate del training, lo rendono particolarmente adatto al controllo continuo in presenza di disturbi stocastici. Tuttavia, questa superiorità è condizionata: se addestrato in ambiente nominale puro, SAC subisce il calo di prestazioni più marcato di tutti i modelli in ambiente perturbato

(−33%, da 9.33 a 6.24 palloncini/ep), evidenziando che la sua efficacia dipende criticamente dall’allineamento tra il dominio di training e quello di deployment.

- **PPO è l’algoritmo più affidabile e coerente:** ottiene risultati leggermente inferiori a SAC ma con un comportamento più prevedibile tra le varianti. Il meccanismo di clipping della funzione obiettivo garantisce aggiornamenti controllati, prevenendo le regressioni catastrofiche che caratterizzano A2C. PPO beneficia significativamente del curriculum learning: la variante curriculum ottiene il punteggio più alto per PPO in entrambi gli ambienti (10.87 nominale, 9.55 noisy) con tassi di crash trascurabili (0% e 0.2% rispettivamente). È la scelta preferibile quando l’affidabilità e la robustezza al trasferimento di dominio sono prioritarie rispetto alle prestazioni assolute.
- **A2C mostra le prestazioni più discontinue.** La sua instabilità strutturale — già emersa nelle curve di training con crolli ricorrenti lungo tutto il processo di apprendimento — si riflette direttamente nei risultati di valutazione: il modello nominale è il peggiore in assoluto (7.43 palloncini/ep, crash 9.0%), mentre il modello noisy risulta sorprendentemente efficace (10.31, 0% crash). Questo suggerisce che A2C beneficia maggiormente di un addestramento su perturbazioni costanti che non di un approccio progressivo, e che le sue limitazioni non derivino dalla difficoltà del compito ma dall’assenza di meccanismi di stabilizzazione degli aggiornamenti.
- **Il PID rimane competitivo in ambiente nominale,** avvicinandosi ai migliori agenti RL (11.02 palloncini/ep, crash 0%), ma degrada drasticamente in ambiente perturbato (5.32 palloncini/ep, crash 12.8%). Il gap di robustezza stimato di circa 5 palloncini/ep è il più elevato tra tutti i controllori confrontati, evidenziando i limiti fondamentali di un approccio basato su regole fisse di fronte a disturbi stocastici non stazionari. In particolare, il rumore sensoriale degrada l’azione derivativa e il vento a random walk sfida l’azione integrale, senza che il PID possa adattarsi automaticamente.
- **La strategia di addestramento ha un impatto sistematico:** addestrare in ambiente perturbato, mediante training diretto (noisy) o progressivo (curriculum), è quasi sempre preferibile rispetto al training nominale quando il deployment avviene in un ambiente con disturbi. Per PPO il curriculum è la scelta migliore; per SAC e A2C il training diretto su ambiente noisy è comparabile o superiore al curriculum. In nessun caso il training nominale puro ha prodotto la variante migliore per il deployment in ambiente perturbato.

7.2 Limiti del lavoro

I risultati presentati vanno letti tenendo conto di alcuni limiti metodologici che ne circoscrivono la generalizzabilità:

- **Singolo seed per configurazione.** Le curve di training analizzate nel Capitolo 6 derivano da singole run per ciascuna configurazione. Sebbene i

risultati di valutazione su 1000 episodi forniscano stime stabili della media, le osservazioni sulla stabilità del training, in particolare i crolli periodici di A2C e il crollo isolato di PPO noisy, vanno considerate indicative e non statisticamente consolidate. La ripetizione su seed multipli (tipicamente 3-5) permetterebbe di distinguere comportamenti sistematici da artefatti casuali della singola run.

- **DQN non riprodotto.** Il valore di riferimento per DQN proviene dalla letteratura [8] e non da un nuovo addestramento in questa codebase. L'algoritmo non è stato incluso nel confronto quantitativo finale poiché la sua limitazione alle azioni discrete lo rende intrinsecamente svantaggiato rispetto agli algoritmi a policy continua, e un confronto diretto sulle stesse metriche potrebbe risultare fuorviante.
- **Budget di training non omogenei.** SAC è stato addestrato per 3.3M step, PPO e A2C per 4M step nelle varianti noisy e curriculum. Questa asimmetria, unita alla differenza di metodologia nel tuning degli iperparametri (grid search per A2C/PPO, manuale per SAC), rende i confronti tra algoritmi parzialmente incrociati: SAC risulta comunque il più performante nonostante un tuning meno estensivo, il che ne sottolinea ulteriormente la robustezza intrinseca, ma un confronto su base omogenea potrebbe modificare il margine relativo tra PPO e SAC.
- **Valori Human indicativi.** I punteggi dell'agente umano riportati nelle tabelle sono stime qualitative basate su sessioni di osservazione nell'arena BalloonEnv.
- **Simulazione 2D.** L'intero lavoro è condotto in un ambiente bidimensionale, che riduce significativamente la complessità del problema di controllo rispetto a un quadricottero reale nello spazio 3D. Fenomeni come il rollio, il beccheggio e l'imbardata simultanei, le interazioni aerodinamiche tra i quattro rotori e le perturbazioni di vento tridimensionali non sono modellati. I risultati non sono quindi direttamente trasferibili a sistemi reali senza ulteriore validazione.

7.3 Sviluppi futuri

I risultati ottenuti aprono diverse direzioni di ricerca e sviluppo:

Estensione a simulazione 3D. Il passo naturale successivo è l'estensione dell'ambiente BalloonEnv a tre dimensioni, introducendo i gradi di libertà aggiuntivi del quadricottero reale (rollio, beccheggio, imbardata) e un modello di vento tridimensionale. Questo richiederebbe una revisione della funzione di reward e dello spazio delle osservazioni, ma consentirebbe di valutare se i vantaggi relativi tra gli algoritmi osservati in 2D si mantengono in uno scenario più realistico.

Controllo multi-agente e scenari swarm. L'arena BalloonEnv supporta già l'esecuzione di più agenti in parallelo. Un'estensione naturale è lo studio di scenari cooperativi o competitivi multi-agente, in cui più quadricotteri devono

coordinare le loro traiettorie per massimizzare la raccolta collettiva di target o per evitare collisioni reciproche. Algoritmi come MADDPG o varianti multi-agente di SAC potrebbero essere esplorati in questo contesto.

Valutazione statistica su seed multipli. Come discusso nella sezione precedente, una limitazione importante del presente lavoro è l'assenza di repliche su seed multipli. Una campagna sperimentale più estesa — anche con budget di training ridotto per contenere i costi computazionali — permetterebbe di quantificare la varianza degli algoritmi e di produrre confronti statisticamente significativi con intervalli di confidenza, rendendo le conclusioni più solide e generalizzabili.

Bibliography

- [1] Yoshua Bengio et al. “Curriculum learning”. In: *International Conference on Machine Learning (ICML)*. 2009. URL: <https://dl.acm.org/doi/10.1145/1553374.1553380>.
- [2] Greg Brockman et al. *OpenAI Gym*. 2016. arXiv: [1606.01540](https://arxiv.org/abs/1606.01540) [cs.LG].
- [3] Alberto Ferrante. *Introduction*. Slides del corso “Edge Computing in the IoT”. Università della Svizzera italiana (USI), piattaforma iCorsi. 2025.
- [4] Tuomas Haarnoja et al. *Soft Actor-Critic Algorithms and Applications*. Tech. rep. arXiv preprint arXiv:1812.05905, 2018. URL: <https://arxiv.org/abs/1812.05905>.
- [5] Volodymyr Mnih et al. “Asynchronous methods for deep reinforcement learning”. In: *International Conference on Machine Learning (ICML)*. 2016. URL: <https://arxiv.org/abs/1602.01783>.
- [6] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518 (2015), pp. 529–533. URL: <https://www.nature.com/articles/nature14236>.
- [7] Antonin Raffin et al. “Stable-Baselines3: Reliable Reinforcement Learning Implementations”. In: *Journal of Machine Learning Research*. Vol. 22. 268. 2021, pp. 1–8.
- [8] Alexandre Sajus. *Reinforcement Learning for the Control of Quadcopters*. <https://github.com/AlexandreSajus/Quadcopter-AI>. Centrale Supélec Project. 2023.
- [9] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017). URL: <https://arxiv.org/abs/1707.06347>.
- [10] Wikipedia contributors. *PID controller* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/PID_controller. [Online; accessed 2026]. 2023.
- [11] H. Zhang and T. Yu. “Taxonomy of Reinforcement Learning Algorithms”. In: *Taxonomy of Reinforcement Learning Algorithms*. Singapore: Springer Singapore, 2020, pp. 125–133. DOI: [10.1007/978-981-15-4095-0_3](https://doi.org/10.1007/978-981-15-4095-0_3). URL: https://doi.org/10.1007/978-981-15-4095-0_3.